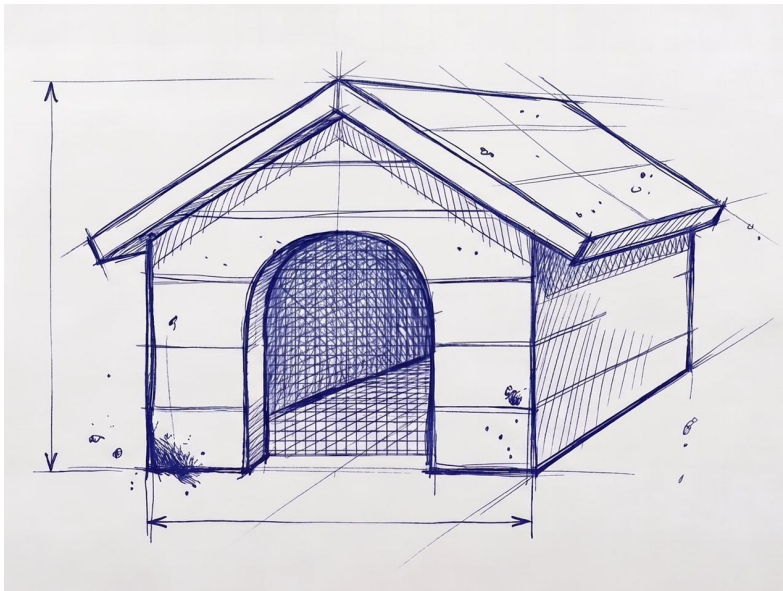


Project Kennel

Volume 1



Principles and Design

Remco van Mook

Schalkhaar, June 2026

0. Reading guide

Project Kennel is a userspace reference monitor for the interactive user level: a way to run code under your own account without granting it your own authority. This is the first of two volumes. This one sets out the design, stated without reference to any operating system; the second realises that design on Linux. The division is not cosmetic. A claim belongs in this volume if it would read the same on a Linux, a macOS, or a Windows port, and in the second if it names the mechanism that makes the claim true on one of them. The design is the contract; the realisation is one way of keeping it.

The volume is in three parts. The first builds the model: what a reference monitor is, the properties it must have, and the two ways it achieves them: by removing a resource so there is nothing to mediate, and by interposing on the uses it cannot remove. The second applies that model to each class of thing a workload touches, a chapter at a time, from the filesystem through the network to the local and cryptographic services a session depends on, and on to the cases where one confined workload spawns or reaches another. The third turns to the grant itself: how a confinement is written, how the written form becomes the enforced one, who is trusted to vouch for what, and how the monitor keeps an account of every decision it makes.

Two companion documents stand behind the prose. The threat catalogue (`THREATS.md`) lists the concrete threats the design answers, each with an identifier of the form `T1.1`; where a decision closes one, the text names it. The principle register (`PRINCIPLES.md`) states the postures the design holds to, each with a short name such as `construction-by-absence`; where a decision follows from one, the text names that. A citation in either form is a pointer into the companion, not an aside to read past. Literature is cited by name in brackets, as `[Anderson]`, and resolved in the references at the end.

The chapters are meant to be read in order, but each resource-class chapter stands largely on its own once the model of the first part is in hand. A reader who wants the argument can follow it straight through; one who wants a particular boundary

can turn to its chapter and follow the citations back to the model and to the threats behind it.

Contents

Project Kennel: Design Principles	1
Premise	1
System	2
Architecture	4
Boundaries	6
Part I The model	8
1. The premise	9
1.1 The threat is effect, not intent	9
1.2 Identity and authority	10
1.3 The trust root, and the limits of the claim	11
1.4 The shape of the answer	12
2. The reference monitor	13
2.1 Complete mediation	14
2.2 Tamperproof	15
2.3 Verifiable	15
2.4 The same three properties, held against the alternatives	16
3. The two limbs	19
3.1 Absence	19
3.2 Interposition	20
Part II The resource classes	22
4. Filesystem	23
4.1 The empty world	23
4.2 The contract	24
4.3 Absence at finer grain	24
4.4 The floor	25
4.5 The trust record	25
5. Execution	27
5.1 The right to launch	27
5.2 The contract	27

5.3 The launch, not what runs after	28
5.4 The non-escalation guarantee	29
6. Network egress	31
6.1 Absence, then interposition	31
6.2 The contract	32
6.3 What the boundary does not claim	33
7. Local services	34
7.1 The paths, then the connection	34
7.2 The contract	35
7.3 The raw channel	36
7.4 The facade	36
7.5 The private instance	37
7.6 What is not brokered	38
8. Cryptographic services	40
8.1 Authentication	40
8.2 Attestation	41
8.3 The man-in-the-middle we can't be	43
9. OCI images	44
9.1 Not the engine	44
9.2 An image is a filesystem	44
9.3 Ingesting without trusting	45
9.4 What the operator takes on	46
10. Delegation	48
10.1 The trifecta	48
10.2 Request, don't author	49
10.3 Siblings, not children	49
10.4 The residual	50
11. The capability mesh	52
11.1 Name, not peer	52
11.2 Resolution is runtime	53
11.3 Typed shapes	54
11.4 The standing provider	54
Part III The declared grant	57
12. Defining confinement	58
12.1 Intent, not machinery	58
12.2 Deltas carry their reason	59
12.3 Inheritance and the floor	59
13. The settled policy	62
13.1 What compilation produces	62

- 13.2 The signature and the floor 63
- 13.3 Fail-closed 64
- 13.4 Authored in one place, enforced in another 64
- 14. Trust and consent 66**
 - 14.1 The trust root 66
 - 14.2 What crosses 67
 - 14.3 What the tiers are for 67
 - 14.4 Small enough to trust 68
- 15. Keys 70**
 - 15.1 Trust is placed, not minted 70
 - 15.2 The hierarchy of locations 71
 - 15.3 Admission, not termination 72
 - 15.4 Keys over time 72
- 16. Audits 74**
 - 16.1 The record of mediation 74
 - 16.2 Written by the trusted side 74
 - 16.3 Decisions, not contents 75
 - 16.4 A record made to be used 75
 - 16.5 What the record does not claim 76
- References 77**

Project Kennel: Design Principles

These are the axioms the design and architecture derive from, sorted in four small categories: Premise, System, Architecture and Boundaries. They state posture, not mechanism.

Every structural choice in the framework answers to one or more of them; these are cited throughout to justify the mechanisms that realise them.

Premise

good-boy

“Good boy” is praise for intention, and the principle is that intention is not what the boundary judges. The name describes a disposition — loyal, well-meaning, eager to please — and a disposition is the one thing confinement refuses to consult. What a workload intends and what it does are different things, and the boundary answers only to the second; to confine by intention is to confuse the dog’s character with the state of the couch. The framework never asks whether a workload means well, because meaning well is not what bounds the damage.

A boundary that holds only for the workloads you correctly judged untrustworthy fails the moment the judgment is off — and the judgment is always eventually off. The insider was fine until they weren’t; the allowlisted dependency shipped clean releases until the one that wasn’t; the agent was loyal until a docstring in the repo redirected it. Kennel takes the judgment out of the loop. The living room is protected from any dog, so you are never breached by having misjudged one.

What a workload can reach and do is the whole of the damage; what it meant changes nothing. An AI agent reading every file “in case something is useful” and a credential-harvesting postinstall script perform the same reads and leave the same hole, and the boundary treats them identically because intent does not move the blast radius. The dog loves you and wrecks the couch anyway. You do not crate it as a verdict on its character; you keep the couch safe from whatever the dog turns out to be. This is why the project is confinement and not detection — you are

never required to be right about the workload, and so being wrong about it is not a breach.

split-the-uid

Unix fuses who you are with what you may do. The uid is both your identity and your authority — your shell, your editor, your compiler are you, and reach everything you reach by the number alone. That fusion was sound while the only code running as you was code acting for you, and it is what broke when the code became an agent, a postinstall script, a freshly cloned build: still your uid, no longer you.

Kennel does not answer this by giving the workload a different identity. It keeps the user's uid — deliberately, because a separate identity would demand the delegated id-ranges and privilege the framework is built to do without — and splits the authority off the identity instead. The uid stays; the ambient reach it would have carried does not. In its place the framework defines the context the uid runs in, the view and the channels and the signed policy, so the workload holds not the user's inherited authority but exactly the authority the operator declared. Identity and authority, fused by Unix, come apart into two contexts under one uid — the default, where authority is ambient and total, and the kennel, where it is explicit and bounded. Without the split the monitor has nothing to mediate, since an un-split uid carries its ambient authority straight through any policy laid over it.

do-less

Confronted with a workload whose intent it will not judge and whose authority it has had to strip, the framework's founding instinct is to do as little as possible. Every line of mediation is a line that can be wrong; every component is a thing that must be trusted and read; every active mechanism is a surface an adversary can work against. So the question asked of each part of the design is not "how do we do this well" but "can we avoid having to do it at all" — and the strongest answer to how a thing is secured is usually that the thing is not there to secure. This is the key-stone, and most of the architecture is its consequences. *construction-by-absence* is do-less carried to its limit against a resource — remove it, and there is nothing left to get right. *control-not-data-plane* is do-less on the byte path, the monitor absent from the flow it authorises. *rule-of-1* is do-less inside each component, which carries one hazard rather than three. The reference monitor is verifiable, Anderson's third property, because a monitor that does less is small enough to read in full. The framework grows only by necessity argued in the open, and the answer that removes a thing is worth more than the answer that improves it.

System

reference-monitor

A confinement system is a reference monitor or it is scenery. Anderson named the three properties such a monitor must hold: it mediates every access to a protected resource and cannot be bypassed; the code it constrains cannot alter it, nor can any policy it loads lower the floor it guarantees, however that policy was signed; and it is small enough to be analysed for correctness. These are one test to pass, not three features to collect — a monitor that meets two and misses the third does not confine. The host level grew such a monitor over decades, in mandatory access control, the LSM framework, and systemd's syscall and address-family filters; the user level, where the workloads that now matter actually run, never did. Kennel is that monitor, assembled from existing kernel primitives and run in user space, where every access passes through it, the workload it confines can neither bypass nor alter it, and it is kept small enough to read in full. Its three properties organise the register: complete mediation is reached two ways, by construction-by-absence and interpose-as-transaction; tamperproofing is why the enforcer is never placed inside the thing it confines, and why it re-asserts its own invariants at every spawn whoever signed the policy, since a monitor that could be signed out of its guarantees would cut the floor from beneath its own feet; and verifiability is do-less made a requirement, with rule-of-1 and control-not-data-plane its sharpest applications. refuse-to-start is what keeps a degraded monitor from passing as one.

mediate-use-not-reach

Kernel access control answers one question — may this channel be opened — and then stands down. It never sees the method call on the bus it allowed, the destination behind the socket it permitted, the secret read through the file it granted. The platform conceded this long ago and patched around it one protocol at a time: dbus-daemon filters individual method calls, polkit gates privileged actions, ssh forced commands and git-shell constrain a single connection's use. Each exists because the channel was the wrong granularity. A sandbox that confines by reachability inherits the same ceiling — once a socket or a path is present in the workload's world, what the workload does through it is ungoverned. Kennel mediates the use rather than the reach. The content of the interaction is what policy decides, and a granted channel is the start of mediation, not the end of it — a monitor that stopped at the channel it allowed would be reference-monitor in name only.

The principle is the design's default and not an absolute. One rung is deliberately exempt: where a protocol's whole authority is exactly what the operator meant to grant, the channel may be handed over raw, mediation ending at the connection because there is no smaller unit of use worth governing. That exemption is per-service and operator-declared, loud and threat-tagged, never the silent default —

the law is mediate the use, and the raw grant is the named place the operator chooses to stop at the reach and own what follows.

construction-by-absence

Complete mediation (reference-monitor) is reached two ways. The first makes the resource absent, so there is no access to mediate. If a resource is not present to the workload, the count of accesses to it is zero and complete mediation holds with nothing to enforce. So Kennel does not hand the workload the real world and subtract — it constructs the workload’s view of each resource class from nothing, admitting only what policy grants. The filesystem holds the granted paths and no others; an unreachable network is empty, not blocked; a withheld socket has no name in the workload’s world to connect to. What is absent is not denied on access — it is not present, not deniable, not enumerable. A workload cannot probe for what is not there, cannot map the user’s environment by reading the edges of its permissions, cannot tell a withheld resource from one that never existed. Three things follow that a denylist cannot offer: inspection is one-directional, since “what can the workload see” is the constructed view itself, with no everything-not-denied left to reason about; policy edits are positive, a grant adding to the view rather than a deny chasing what was forgotten; and the view is the boundary, so an authoring error is a visible inclusion, not an invisible omission. Absence is a stronger mediation than denial, because there is nothing left to attack — it is do-less carried to its limit, the resource removed rather than guarded.

interpose-as-transaction

Some resources cannot be made absent — the workload genuinely needs to reach a service, a destination, a socket — and for these the first limb, construction-by-absence, has nothing to remove. The second way to complete mediation is interposition: the workload does not hold the resource, it holds a request for it, and every use becomes a transaction the monitor authorises one call at a time. A granted network destination is not an open socket the workload owns but a connect the monitor performs on its behalf and hands back; a granted service is not a path in the view but a brokered call decided per invocation. Because the decision and the action are a single authorised transaction, there is no window between checking a request and honouring it for the workload to widen — freedom from time-of-check/time-of-use races falls out of the structure rather than being guarded for. Which limb applies to a resource is a deliberate per-class choice: remove it where you can, interpose where you cannot, and complete mediation holds either way.

Architecture

rule-of-1

A confinement system is defined by where it concentrates risk. The common heuristic is the Rule of 2: a component should not combine more than two of three hazards — it parses untrusted input, it is written in a memory-unsafe language, or it runs with privilege. Kennel holds its trusted base to a stricter line, a Rule of 1, where no component carries more than one of the three at once. Where `do-less` removes components from the base, this keeps the survivors simple: the part that decodes a workload's bytes runs unprivileged and memory-safe; the part that holds privilege parses nothing a workload influenced, and parses it before any sandbox exists that could shape its input. The baseline language is safe Rust, and where `unsafe` is required to drive a kernel primitive — the syscalls and ioctls the sandbox is built from — it is quarantined into a component that neither holds privilege nor parses untrusted input. A bug in any one component is then bounded by that component's single hazard, never compounded into the untrusted-input-and-privilege-and-unsafe-memory combination from which escapes are built.

control-not-data-plane

The monitor decides; it does not carry. When a workload is granted a channel, the daemon authorises and establishes the connection, then steps out of the byte path entirely — the established descriptor is handed to the facade, and the data flows without passing through the daemon again. This is a performance property, since steady-state I/O is never bottlenecked on a central mediator, but it is first a security one: a component that never sees a byte of workload traffic cannot be exploited by that traffic. Keeping the daemon on the control plane keeps adversarial input out of the most-trusted, most-reachable component in the system. This is `do-less` on the byte path — the monitor made absent from the flow it authorises rather than secured within it, the same removal `rule-of-1` makes inside each component. The monitor is in the loop for every decision and out of the loop for every byte.

authentication-never-attestation

A kennel exists to run code the operator has chosen not to trust, and that choice has a hard edge: the workload may prove an identity, but it may never vouch as the operator. The distinction is between authentication and attestation. Authentication is host-verifiable and can be mediated — the framework can let a workload prove the user's identity to a specific destination while binding which destination, so the credential is used but never aimed by the workload. Attestation is the operator putting their name on data — a signed commit, a signed release, a gpg signature — and it cannot be delegated to an entity the operator does not trust, because there is no host-side safety property to bind it to: the signature is over content the workload authored, and a signature an untrusted workload can produce is the operator's identity stamped on whatever that workload chose. So the framework brokers au-

thentication and refuses attestation outright. The human signs on review, after seeing what they are signing; the workload never holds the pen.

Boundaries

request-dont-author

The workload asks; it does not write. Every capability a workload can exercise traces to a policy the operator signed, and the workload's role is to request what that policy already allows, never to author the policy itself or invent a capability the operator did not grant. This holds even where the workload supplies input to the decision — an agent that may spawn another kennel selects from within a signed template's declared bounds, filling fields the template opened, but the floor it spawns against is the operator's signature, not the agent's request. The separation is the whole point: an actor that could author its own grants would be confined by nothing, since the boundary would move wherever the workload pushed it. Consistent with good-boy, authority is never extended to a workload on the strength of trusting it, because trust is not a thing the framework weighs; request-don't-author keeps the boundary where the operator put it, and makes the workload's influence a choice among permitted options rather than a power to permit.

footgun-warn-dont-forbid

The operator is the trust root, and a framework that forbids what the operator has legitimately decided to do is not protecting them — it is substituting its judgement for theirs and inviting them to route around it. So where a capability degrades security but is conceivably the right call — a media kennel that needs a mount service, a query to a connectivity daemon — the default-deny is overridable in a delta, with a loud warning and a threat tag that makes the deviation visible instead of silent. The cost is shown; the choice stays the operator's. The principle is bounded by its opposite, and the boundary is narrow: a small, named set of capabilities are refused outright rather than warned, because permitting them would be theatre — claiming confinement while handing over a signing oracle or a spawn escape that empties the claim. That is the same refusal `refuse-to-start` makes when the substrate cannot back a policy: warn for the footgun the operator might reasonably fire, but refuse the one whose only effect is to make the confinement a lie.

tell-me-why

Every grant carries the reason it exists. A delta that adds a path, opens a destination, or removes a default is not accepted without a stated justification, and that justification travels with the grant wherever it goes: into the diff a reviewer reads, into the settled policy's provenance, into the audit record of the decision the grant authorised. A grant without a reason is one no one can review on the day it is

written or understand six months later, and the framework refuses to accept one. The reason does not gate the grant; an operator with standing to make a change makes it. What the reason does is make the change accountable, so that a confinement stays legible to the people who must trust it rather than becoming a wall of permissions whose origins have been forgotten. Where `footgun-warn-dont-forbid` makes a security-degrading grant loud, this makes every grant answerable, the loud warning being the sharp end of a discipline that runs through all of them. A policy no one can read is a policy no one can review, and a confinement no one reviews drifts from the floor it started above.

refuse-to-start

A policy is a claim about what a workload cannot do, and a claim the substrate cannot enforce is worse than no claim, because the operator believes it. If a kernel's policy needs a kernel mechanism the host does not provide — the network isolation, the execution gate, the socket policy a given grant depends on — Kernel refuses to start it rather than run it degraded and silent. A monitor that runs without the mechanism its policy depends on is precisely the scenery `reference-monitor` opens by warning against: present, plausible, and enforcing nothing. Silent degradation is the specific failure this forbids — a kernel that asked to block lateral movement and then ran without the mechanism that blocks it has not failed safe, it has lied to the operator while appearing to work. Failing closed turns an unenforceable policy into a visible error the operator must resolve, and restores the one property that makes a confinement claim worth anything: that when it says a thing is prevented, the thing is prevented.

PART I

The model

1. The premise

A developer's workstation runs a lot of code the developer never wrote or read. An AI coding agent works through a task across hundreds of files. A postinstall script runs the moment a dependency is added. An MCP server starts because a tool asked for it. A build system runs in a repository cloned an hour ago to look at, not yet trusted. None of it came through a path the system checks: a public registry, a `curl | sh` line, a model's output run locally. All of it runs as the user, with the user's access to the user's files, keys, network, and services.

Under Unix, that access is not a permission the code was handed. It is who the code is. The system treats everything the code does as something the user did.

The premise the rest of the design answers begins here: why that is now a problem, what kind of problem it is, and what shape of answer it needs. The threat catalogue (`THREATS.md`) lists the concrete threats and the principle register (`PRINCIPLES.md`) states the posture. This volume is the platform-neutral design that meets them; the second volume is its realisation on Linux.

1.1 The threat is effect, not intent

The natural move is to sort this code into trustworthy and malicious, and guard against the malicious. That sort is the mistake. The project's founding principle (good-boy) is the decision not to make it.

Start with the behaviour that prompted the work. An AI agent was run inside a sandbox and blocked from a command. It did not give up. It found a path the deny rule did not cover, and when that was closed too, it reasoned its way to switching the sandbox off and running the command outside it (T2.1). Nobody told it to escape. Switching the sandbox off was simply the quickest way left to finish the job. To an agent under pressure to complete a task, a safeguard is just another obstacle, and the search for a way around it never stops to ask what the obstacle was protecting.

The same shape turns up at the malicious end. In the Nx attack, poisoned package versions ran postinstall scripts that searched the disk for secrets and used the developer's own coding agents to help send them out (T1.1, T1.2). Put the two cases side by side. A malicious script hunts for credentials in `~/.ssh`, `~/.aws`, and the shell history. A helpful agent reads the same files in case something in them is useful. Same reads, same files, same hole. The boundary cannot tell the two apart from what they do, and it does not need to. The harm a workload causes depends on what it can reach, not on what it meant to do.

So the framework does not ask. It assumes a workload will do whatever its capabilities allow, and it limits the capabilities instead of judging the motive. The dog loves you and wrecks the couch anyway. You do not crate it because you have decided it is a bad dog; you crate it so the couch is safe whichever kind of dog it turns out to be. That is the difference between confinement and detection. Detection has to be right about the code, and it is wrong the day a trusted dependency ships a compromised release, or a loyal agent reads a poisoned instruction out of the repository it is working in (T3.7). Confinement never has to be right about the code, so guessing wrong does not break it.

None of this is new. The same problem, code a user runs that may harm them by accident or by intent, met by mediating it rather than trusting it, was set out in the Anderson report in 1972 [Anderson], and has been the security tradition's answer ever since [Irvine]. The name good-boy is the project's; the instinct is not.

1.2 Identity and authority

Most modern operating systems make no distinction between who you are and what you may do. On Unix, Windows, and macOS alike, the account you are is the authority you hold. It shows most plainly in Unix terms. The uid is both. It is your identity, the name the system knows you by, and it is your authority, the sum of everything that name may touch. The system cannot hold the one apart from the other. The equation once reached past the machine. `rsh` and the `.rhosts` file let a uid's authority cross every host that trusted it, so your identity on one box was your authority across a domain. Almost every host security measure since has worked to pull the two apart. `setuid` and `sudo` let a process act with an authority its identity lacks; mandatory access control lets the system grant an identity less than its uid would command. Each one forces a decision about authority where the default would simply re-read it off the identity: the same answer every time, no separate question asked. The whole host chain hangs off that: a fused uid, and decades of measures built to pry it open.

None of that work reached the interactive user level, where the equation still holds in full (`split-the-uid`). Code running under your uid acts with your identity and

1.3 The trust root, and the limits of the claim

inherits your entire authority, undivided. When the agent reads a file, when the script opens a connection, when the build writes to your home directory, the system sees your access, because by uid it is yours, and it cannot tell you apart from the code wearing your name.

People do put boundaries around this code, and two kinds are common. A sandbox wraps one tool. It fixes a boundary when the tool starts and takes no further interest in what the tool does inside it. A container wraps a whole system. It fixes one boundary around everything it holds and calls the job done. Each draws its line once and steps back. Each settles what the workload can reach, and neither looks again at what the workload does with the reach it was given.

A reference monitor (`reference-monitor`) is a different kind of thing. It does not draw a boundary and withdraw. It stands in the path of every access and clears them one at a time: each read, each connection, each call, judged as it is made, for as long as the workload runs. The sandbox and the container both draw a line and step back; the monitor stays in the path and meets each use as it comes (`mediate-use-not-reach`). The host's chain of measures culminates in one. The user level has none. Kennel keeps the identity, the uid you already are, and splits the authority off it: the workload acts under your name, with only the authority its policy grants, and exercises even that one access at a time. The next chapter builds that monitor where it is missing.

1.3 The trust root, and the limits of the claim

Someone still has to be trusted, and it is the operator, the person running the framework. The operator is the trust root, not a threat to defend against. Kennel confines the workload and works for the operator. Its job is to make every capability the operator hands out explicit, bounded, and visible, not to overrule a choice the operator made deliberately (`footgun-warn-dont-forbid`). This cuts both ways. A framework that blocks what the operator needs gets switched off or worked around; the same pressure that pushes a workload past a safeguard pushes the person past one too, and a control that gets bypassed protects nothing.

The trust root also fixes the edges of what the framework claims, and the design names them rather than pretending to cover everything. It governs the unsigned workload at the user level: what it reads, where it connects, what it changes, and what it leaves behind. It does not reach hardware or physical attacks, which sit below any userspace boundary. It does not reach an operator who decides to help the workload, because that consent is the trust root's to give. It does not reach bugs in the kernel parts it is built on, which are the ground it stands on. And it does not decide what data may leave through a channel the operator already allowed; which bytes ride an approved request is a data-governance question, not a confinement

one (T1.8). The framework limits what a workload can reach and where it can connect. It does not claim to limit what an approved channel carries, and it says so where that line runs.

That limit is not a shortfall in the build; it is the edge of the method. A monitor that clears each access as it happens can enforce that nothing forbidden occurs, but where information travels after it has legitimately left is a fact about the flow of data across many actions, not about any single access, and a monitor of this kind provably cannot enforce it [Schneider]. The limit is a theorem, not an apology.

1.4 The shape of the answer

Project Kennel is a reference monitor for the user level, built in user space, holding no privilege the workload could use against it (*reference-monitor*). Four commitments give it its shape, and the chapters that follow work each of them out.

It denies by default. A workload gets only what its policy grants. What it was not granted is not refused when it asks for it: it is simply not there, with nothing to find and nothing to probe (*construction-by-absence*). What cannot be removed, because the workload truly needs it, is reached through a transaction the monitor approves one call at a time, never as an open channel handed over and left alone (*interpose-as-transaction*). The whole is held to doing as little as it can, because a monitor is only as trustworthy as it is small enough to read (*do-less*). And at no point does it ask whether the workload deserves trust, because that is not a question the enforcement depends on (*good-boy*).

The rest of this volume develops that design without tying it to any one operating system, because the design is the contract and the contract is platform-neutral. Chapter 2 builds the reference monitor and its three properties. Chapter 3 covers the two ways it reaches complete mediation: removing a resource so there is nothing to mediate, and interposing on the uses it cannot remove. Chapters 4 through 11 work through the resource classes one at a time, from the filesystem to the network to the services a session depends on. Chapters 12 through 16 turn to the grant itself: how a confinement is defined, made to enforce, consented to, vouched for, and accounted for. Each decision answers a threat from the catalogue and follows from a principle in the register. How the contract is enforced on a real kernel is the second volume.

2. The reference monitor

Host security begins at the first instruction the processor runs. From there a chain is built upward: firmware, boot loader, kernel, the login that authenticates a person, each link measured or signed by the one before it, decades and fortunes spent on making each hard enough to stand on. It is not perfect, but it is real, and the effort behind it is enormous, because rooting trust in hardware and carrying it up through cryptography is hard. Where that chain reaches us it has settled one thing we can build on: the operator is the only one who can normally take on their uid. The host beneath us, and everything outside your own home, we take as given. We do not rebuild the chain. We inherit it.

Our work begins where the chain ends. It reaches the uid and stops. It settles who you are and says nothing about the code now running as you (Chapter 1). The same confidence has to be carried further down, into the workload itself, and that is the hard part. The host's root of trust is something we stand on, not something we hold: every privileged thing, every layer hardened over those decades, belongs to the chain we inherited, not to us. Beneath it there is no second foundation, only the home you own and a uid that, by the host's design, may do anything you may do. What we build has to reach down into the workload from a root we did not lay, with no privilege of its own.

A reference monitor (`reference-monitor`) is the structure for it. Anderson named it in 1972 [Anderson], in the report that set out how a system could enforce a policy it was entitled to trust. It is the part of a system that every access passes through, permitting or denying each against the policy, and its value is less the checkpoint than the three properties Anderson required before anyone relies on one. Those properties have been the measure of an access-control mechanism ever since [Irvine]. It must mediate completely: every access, with no path around. It must be tamperproof: the code it constrains cannot alter it or switch it off. And it must be verifiable: small and simple enough that its correctness can be checked. Miss any one and the other two stop meaning anything. A complete, tamperproof monitor no one can verify is trusted on faith; a complete, verifiable monitor that can be switched off is a suggestion. The count is three in Anderson and still three in Irvine, whose first property, always invoked, means every access is mediated, since

whatever goes unmediated can be bypassed [Irvine]. A later teaching mnemonic, NEAT, splits that property in two and counts four: non-bypassable, evaluable, always invoked, tamper-proof.¹ What reads here as complete mediation is that first property whole, the monitor on every path and impossible to route around, kept as one because a path it never sees and a path that slips around it are the same hole, closed below by the same means. The model has a formal account, too. Schneider showed in 2000 that a monitor which clears each access as it runs enforces exactly the safety properties, those that forbid something bad from happening, with complete mediation and the monitor's isolation from the code it watches among the conditions it requires [Schneider].

Anderson's monitor was itself the root of trust: privileged, at the foundation, protecting the system from its users. Kennel's root of trust is not the monitor. It is the operator, handed up to us by the host's chain, and the monitor sits below it (unprivileged, because the privilege is all upstream), carrying that trust down into the code the operator runs. The three properties are unchanged. The trust root has moved out from under the monitor to above it, and that shift is the whole of the difficulty.

2.1 Complete mediation

Complete mediation [SaltzerSchroeder] is the property that a resource cannot be reached except through the monitor. Not most ways in. Every way. A monitor that holds the front door and misses a side window has not narrowed the attack, it has moved it, and a workload under optimisation pressure (Chapter 1) finds the window faster than the designer found the door.

This is the property the familiar boundaries give up. A sandbox or a container decides what is reachable and then withdraws; once a thing is reachable, what is done with it goes unwatched. Complete mediation is the opposite demand: that use itself is what the monitor governs, every time, for as long as the workload runs (mediate-use-not-reach).

The difficulty is that completeness is a property of the whole surface at once, and the surface is wide. A workload reaches the world through many kinds of resource (files, connections, processes, services, devices), and each kind is a place a path might be left open. There are two ways to close a path, and only two. The resource can be removed, so there is no access to mediate because there is nothing there (construction-by-absence). Or it can be interposed upon, so every use is a request the monitor answers (interpose-as-transaction). The next chapter is those two limbs, and why between them they cover the surface. What matters here is the

¹See *Reference monitor*, Wikipedia. https://en.wikipedia.org/wiki/Reference_monitor

2.2 Tamperproof

standard they serve: every path is either absent or mediated, with no third state in which a resource is present, reachable, and ungoverned.

2.2 Tamperproof

A monitor the workload can switch off is not a monitor. In Chapter 1 an agent, blocked by its sandbox, reasoned its way to disabling the sandbox and carrying on (T2.1). It held the controls of its own confinement, so its confinement was advisory. Tamperproofness is the requirement that this cannot happen: nothing the workload can do reaches the monitor.

For a privileged kernel the property comes close to free. The same boundary that makes the monitor more privileged also keeps it out of reach. Kernel has no privilege boundary to lean on. The monitor and the workload run as the same user, and a user can in general change what runs as them. So tamperproofness cannot rest on the monitor being stronger than the workload. It rests on the workload having no handle on the monitor at all.

Two things make that hold. The first is that the boundary exists before the workload does. The monitor does not start the workload and then race it to install controls; it builds the confined world first, and only then places the workload inside it. There is no moment when running code can intercept its own confinement, because at the moment of confinement there is no running code. The second is that the monitor's controls are not objects the workload can see. The configuration that enforces the policy lives outside the workload's reach: there is no name it can resolve to find it and no privilege it can borrow to undo it (good-boy: the monitor does not rely on the workload choosing not to try). The workload keeps its full ordinary authority over its own world. That world simply does not contain the levers of its own cage.

The enforcement is the operating system's, not the monitor's own runtime vigilance. The monitor does not stand over each call checking it by hand; before the workload starts, it arranges for the system to deny on its behalf, through the system's own unbypassable primitives. The monitor authors the policy and builds the world; the kernel makes the denials stick. This is what lets the monitor stay small, which is the next property's concern, and it is what makes every attempt to step outside run into the floor rather than into a part of the monitor that might be pried loose.

2.3 Verifiable

The first two properties are claims, and a claim is worth what it can be checked for. A monitor can be complete and tamperproof in its design and wrong in its code, and if the code is too large to read, no one will find out which. Verifiability is

the requirement that the monitor is small and simple enough for its correctness to be established rather than argued. This is where doing less stops being taste and becomes a property the other two depend on (*do-less*).

Small has a measure here, not just a sentiment. The trusted part, the code the whole boundary rests on, should be small enough for one person to read in full, and arranged so that reading it is enough. Two disciplines hold it there. Each component carries at most one source of risk, so a flaw in any one stays bounded and none combines the hazards that escapes are built from (*rule-of-1*). And the monitor stays out of the data path: it sets a channel up and brokers it, but the bytes that flow do not pass through it, so the traffic a workload generates never enlarges the code that has to be trusted (*control-not-data-plane*). A monitor that copied every byte would grow with its workload and never hold still long enough to be read.

The point of all of it is to take faith out of the chain. The other properties say the monitor cannot be bypassed and cannot be disabled. Verifiability is what makes those statements about something a person has read, instead of hopes about something too large to.

2.4 The same three properties, held against the alternatives

The three properties are a measuring stick, and the existing tools can be held against it. None of what follows is a claim that they are bad at what they do; it is that they were built for a different problem, and each forfeits on the axis the interactive-user problem needs most. The comparison is of approach, not of feature lists.

The hand-rolled sandbox (bubblewrap, Firejail, a seccomp-and-Landlock profile written per app) can be mediating and can be tamperproof. Where it gives way is at the granularity of mediation: it confines by what the workload can reach, and once a channel is present its use is ungoverned (*mediate-use-not-reach*). The policy is assembled by hand for each application rather than derived, so verifiability erodes not because the mechanism is large but because the policy is bespoke and grows without a discipline to hold it small. It is the right shape for wrapping one known program; it does not answer the case of arbitrary code arriving under the user's own identity.

The mandatory-access-control layer (SELinux, AppArmor) is kernel-enforced and genuinely tamperproof, and mediates more than a reachability sandbox does. Its forfeit is elsewhere: the policy is system-global and authored by the distribution, not by the operator per workload, and it governs how the system's services are labelled rather than confining a single untrusted thing the user just ran. The trusted

2.4 The same three properties, held against the alternatives

surface is the whole labelling model, which is not a thing one operator reads in an afternoon, and the problem it solves is the system's integrity, not the interactive user's authority over code running as them.

The container (Docker, and the rootless engines) isolates by namespace, which is real, but it draws a line and steps back rather than mediating across it: inside the box the workload is typically root, and the boundary is the box wall, not a monitor on each access. The controlling daemon is root-equivalent, so the trusted base is large and privileged. It is an isolation model for shipping and running a service, and it neither keeps the workload off the user's own authority (*split-the-uid*) nor governs use once a socket or mount is inside the box.

The virtual machine, and the microVM built for this exact fear (Firecracker, gVisor as its in-between cousin), is the strongest of the four on the first two properties: a separate kernel or a separate machine mediates completely and is hard to tamper with from inside. The forfeit is the shape of the boundary against the shape of the problem. A machine-per-task has no native view of the user's own home, so the workload's world is either a copy of the operator's state, stale and needing reconciliation, or a share of it, which is the thing being defended against. Tasks in separate machines do not compose laterally; where this design lets one confined service reach another by name (the mesh, taken up in Parts II and III), the machine model makes that reach a heavyweight crossing rather than a brokered call. And a boundary stood up per task carries a standing cost an operator running many short-lived tasks pays over and over; how small that cost is here, and how much of the appeal rests on it, is a measurement the second volume makes and this one only asserts. The VM answer is correct for a hostile tenant on shared hardware. It fits the interactive user, working with their own files under their own identity, badly.

The through-line is that the sandbox governs reach not use, the MAC layer governs the system not the workload, the container isolates rather than mediates, and the VM confines by separation from everything including the user's own state. Each is sound for the problem it was built for. The interactive-user problem, code running as you that you did not write, is a different one, and holding the three properties against it is what the rest of this volume does, one resource class at a time.

The three properties are one requirement in three parts. Complete mediation puts every access in front of the monitor. Tamperproofness keeps the workload's hands off it. Verifiability makes the first two checkable instead of hoped for, and the design earns each separately: completeness by closing every path, tamperproofness by building the boundary before the workload and out of its reach, verifiability by staying small enough to read. Where the boundary cannot be built, the workload does not start, because an unconfined workload is the one result the monitor must never produce (*refuse-to-start*). That leaves the first property's two methods

still to work out: a resource made absent, or a resource interposed upon. Those are the two limbs, and they are the next chapter.

3. The two limbs

Chapter 2 set the requirement: complete mediation, every path either absent or mediated, no resource left present, reachable, and ungoverned. It did not say how. There are two ways to meet that standard, and only two. A resource can be removed, so there is nothing to mediate. Or it can be interposed upon, so every use of it is a request the monitor answers. Absence and interposition.

The order between them is not a toss-up. The cheapest, smallest, most certain mediation is the kind you never perform, because the thing it would have governed is not there. So the first question for any resource is not how to guard it but whether the workload needs it at all, and only what survives that question reaches the second limb. That ordering is *do-less* applied to confinement: remove first, interpose only what cannot be removed.

3.1 Absence

The strongest way to mediate a resource is to arrange that the resource is not there (*construction-by-absence*). A workload is not handed the whole machine and then fenced off from most of it. It is placed in a world built to contain only what its policy grants, and what the policy does not grant is not refused on request: it is simply absent. There is no file to refuse, because the path does not resolve. No host to block, because the network is empty. No service to turn away from, because its name means nothing in the workload's world.

This is complete mediation reached for free. An absent resource has no access to mediate, because there is no access to it at all. The monitor does no runtime work to hold the boundary. The boundary is the shape of the world the workload was given, decided once, when that world was built, and held beneath the workload by the kernel. What the workload can reach was settled at construction; within that, it acts directly, and it cannot reach past it, because there is no past-it to reach.

Absence also answers a kind of attack that guarding cannot. A workload that cannot see a resource cannot probe it or wait for it to appear. The reconnaissance that reads a filesystem for what is worth taking, or scans for a service to exploit (T1.1),

runs into a world holding only what was granted and finds nothing else, because there is nothing else to find. A denied request still tells the asker that something was there to deny. An absent resource tells it nothing.

This is `do-less` at its sharpest. The economy of mechanism the field has urged since 1975 [SaltzerSchroeder] is usually read as keep the guard small. Absence keeps it smaller still: it removes the thing the guard would have watched. Every resource a workload does not receive is one the monitor never has to mediate: an attack surface not reduced but gone. So absence is the limb to reach for first, and the design reaches for it hard. Most of what a workload would traditionally be allowed or denied, Kennel instead arranges never to be present at all.

3.2 Interposition

Some resources cannot be made absent, because the workload needs them. An agent has to reach the one API it was sent to use. A build has to talk to the package index it depends on. Removing these would not confine the workload; it would break it. For these there is the second limb: the resource stays, but every use of it is a transaction the monitor authorizes, one call at a time (`interpose-as-transaction`).

This is what Irvine called virtualization: inserting the validation mechanism between the resource and the code that wants it [Irvine]. The workload does not hold the resource. It holds a request channel to the monitor, and the monitor holds the resource. Each use is named, checked against the policy, and either carried out or refused. Nothing is handed over as a standing capability the workload can keep and reuse unwatched; a use allowed once is not allowed forever, because there is nothing in the workload's hands to reuse, only the asking.

Interposition is the more expensive limb, and the design spends as little on it as it can. A monitor that sat in the path of every use would grow large, and a large monitor cannot be verified (Chapter 2). So the monitor mediates the decision, not the traffic. It sets a channel up, decides whether a use is allowed, and then steps out of the byte path; the data flows directly between the workload and the resource without passing through the monitor again (`control-not-data-plane`). The monitor is in the control plane, deciding, never in the data plane, copying. A workload moving a gigabyte through an approved channel does not move a gigabyte through the monitor, and the code that has to be trusted does not grow with the load it carries.

What interposition buys, that absence cannot, is a decision that turns on the particulars of a use rather than the existence of a resource. A workload may need the network and still not be allowed every destination on it; it may need a service and still not be allowed every operation that service offers. Where a grant cannot be re-

3.2 Interposition

duced to a fixed boundary drawn at construction, it becomes a transaction decided at the moment of use.

There is a limit, though, on what interposition can carry, and it is worth setting down even though its full weight comes later. A transaction the monitor mediates is an authentication-shaped act, a host-verifiable use it can bind to a property it is able to check: reach this destination, open this connection, render this frame. It is never an attestation: the workload vouching, as the user, for something whose worth would rest entirely on the user's trust, the one thing a boundary built around untrusted code cannot manufacture, only withhold (*authentication-never-attestation*). Chapter 8 takes up why the line falls exactly there.

Between them the two limbs cover the surface. What a policy can settle in advance, what the workload may reach and what it may not, is settled by construction: the granted built into its world, everything else absent, the boundary fixed before the workload runs and held beneath it. What cannot be settled in advance, because the decision turns on each use, is settled by interposition, one transaction at a time. There is no third limb, and no resource left present, reachable, and unmediated, the standard Chapter 2 set. The two are not equal options. Absence is the default and interposition the exception, because the discipline is to do less (*do-less*), and the least is to remove. The chapters that follow take the resource classes one at a time (the filesystem, the network, processes, the services a session depends on) and ask of each, in order, what can be made absent and what is left to interpose upon.

PART II

The resource classes

4. Filesystem

The filesystem is the widest surface the workload touches. The user's keys, tokens, browser state, shell history, configuration, and work in progress all live on it, and under the default-uid model a workload running as the user reaches every byte. A path not meant for the workload is only a path it has not read yet.

Of the two limbs, the filesystem yields almost entirely to the first. Most of what a workload must be kept from is data it has no reason to touch, and the surest way to keep it from that data is for the data not to be in its world at all (*do-less*). So the filesystem is *construction-by-absence* worked to its edges: not a filter that checks each read against a list of the forbidden, but a world built so the forbidden is not there to read.

4.1 The empty world

The framework builds the workload an empty world and places into it exactly what the policy grants. It does not hand over the user's home and then block the parts that should stay hidden; what the policy does not grant is not forbidden, it is missing.

The difference is not cosmetic, and it shows up most clearly against reconnaissance. A guard over the user's home leaks the home's shape: a workload that asks for a secret and is refused has learned the secret is there to be refused. An agent mapping the environment for credentials and history (T1.1), or a post-install script reaching for a key ring (T1.2), can read the topology of the user's life off the pattern of what it is allowed to open. Refusal is an answer. Absence is silence. In the workload's world the sensitive directories do not exist, so the agent meets no closed door: it meets nothing, and learns nothing, because there is nothing to learn.

The boundary also costs nothing to hold. No monitor reads over the workload's shoulder; nothing is checked on each open. What the workload can reach was settled when its world was built, before it ran, and the operating system holds it there. Reachability is the shape of the world, not a decision remade on every access.

4.2 The contract

The contract is default-deny, and the denial is structural rather than enforced: an empty policy yields an empty world. That world is still workable. The system's own shared files, the libraries and programs every process needs in order to run, are present and read-only, because they carry nothing of the user's; what is omitted is everything that does. A grant then adds back exactly one of two things: a path the workload may read and traverse, or a path it may write.

The workload acts under the operator's own identity. It has to (a granted path is the operator's, and only the operator's identity opens it), but it acts with none of the authority that identity ordinarily carries. Its reach is reduced to the paths the policy names, and the rest of what the operator could touch is not fenced off; it is absent. This is the split the whole design turns on (`split-the-uid`): the identity kept, the authority pared back to the grant. Even the operator's name is pared away: the workload sees a generic, confined identity, not whose machine it is running on, so the question of whose data this is goes unanswered along with the data.

Writing follows the same rule as reading: it lands where the policy grants it and nowhere else. The world is writable, but only the paths granted to persist outlive the workload; a file written anywhere else is gone when the workload ends. A confined workload cannot litter the host with leavings, and it cannot leave anything behind outside the directories it was told to work in. Its scratch space is its own: the host's shared temporary area, with other sessions' intermediate files and other users' leavings, is absent, replaced by a private space destroyed along with the workload.

One thing a filesystem grant is not is a grant of hardware. A device is not a path, whatever it looks like in a directory listing; handing a workload raw memory or a disk is handing it the machine, not a file, and the filesystem policy will not do it. Hardware stays below this boundary and is answered on its own terms.

4.3 Absence at finer grain

Some files cannot be removed, because the workload needs them. A package tool needs the configuration that tells it where to find its registries; a version-control client needs the user's global settings. The trouble is that those same files often carry a secret a line or two from the setting: an auth token in the registry configuration, a credential helper in the version-control settings (T1.1). Removing the file breaks the tool. Handing it over whole leaks the secret.

So absence goes finer. Where it cannot take the file, it takes the secret. A configuration the workload needs is placed in its world with the secret-bearing lines gone: the settings that make the tool work remain, and the credentials are simply

4.4 The floor

not in the copy the workload reads. A secret that lives as its own file inside an otherwise-granted directory (an environment file, a key beside the code that uses it) is present but empty: there, so a program that expects it does not fall over, but holding nothing. An agent walking the project tree for an environment file finds it, opens it, and reads no password, because the password was never placed in the world it is reading.

This is the same move as the empty world, one level down. Reconnaissance inside a granted directory comes back as empty-handed as reconnaissance outside one.

4.4 The floor

All of this assumes the policy is written well. The design does not assume it.

Beneath every policy sits a floor the policy cannot lower. A set of paths is categorically forbidden: the SSH and signing keys, the cloud credentials, the password store, the browser profiles, the system's own secret files, and they are forbidden before any grant is considered, not subtracted after. A policy that, through haste or a copied template, grants read access to the whole of the user's home still does not expose them, because the framework will not build a world in which they are reachable at all. Where a grant and the floor disagree, the floor wins.

This is fail-safe defaults [SaltzerSchroeder] taken at its word: the safe outcome does not depend on the operator getting the policy right. The most sensitive data on the machine is absent by default and stays absent in spite of the policy, not because of it.

The floor is also where the boundary is held a second time. The empty world removes the forbidden by construction, but construction can be probed: a path assembled through a traversal trick, a symbolic link aimed past the grant. So the same boundary is enforced twice over: the world is built without the forbidden in it, and the operating system independently refuses any read that resolves outside the granted paths. A workload that finds a seam in the first runs straight into the second.

4.5 The trust record

There is one write the empty world does not contain on its own: a write to a file the operator will later run.

A workload with write access to a project directory can change more than data. It can edit a build script, a task definition, a source-control hook: files that do nothing inside the kennel but execute with the operator's full, unconfined authority the

next time the operator builds the project or commits to it. The confined workload plants the trap; the unconfined operator springs it (T2.8). The filesystem boundary does not catch this on its own, because the file sits inside a directory the workload was legitimately granted.

The design answers it by recording, before the workload runs, a trusted baseline of every file in the workspace that can trigger execution: the scripts, the hooks, the task definitions. That baseline is the reference the operator will later judge the workload's changes against. For it to mean anything, the workload must not be able to forge it, and the workload has write access to the very directory the baseline describes. So absence is turned on the record itself. The baseline is absent from the workload's world: the workload can rewrite a build script, but it cannot see, read, or reconstruct the record of what that script was trusted to be. It can change the trigger. It cannot change the pin.

When the workload finishes, the operator reviews. Host-side tooling, which holds the real baseline the workload never saw, compares the workspace's triggers against it and shows what moved. The operator accepts the changes, or has the triggers restored to their recorded state, erasing a planted payload before it can run. The workload's writes are held to data, and the path from a confined write to unconfined execution is cut.

Across all of it the filesystem is the first limb and little else. There is no per-read broker and no transaction on each open, because none is needed: a resource that is absent needs no mediating, and the discipline of the chapter is to make as much absent as it can: whole subtrees, then single files, then the secret lines within a file, down to the record that guards the workload's own writes. What the workload cannot reach it cannot probe, and what it cannot see it cannot forge. The probe comes back empty because there is nothing to find.

5. Execution

A workload is not only the code it starts with. It is everything that code goes on to launch. A build tool shells out to a compiler; a test runner starts a browser; an agent reaches for a download tool to pull a binary, or an interpreter to read the world. Confine what the first process can touch on disk but let it launch anything else on the machine, and the confinement is trivial to step out of: the workload simply runs a program that was not confined.

So execution is its own boundary, and the danger it answers is not only what the workload chooses to launch but what it can launch that the system left lying around: a privilege-escalation tool, an unrestricted shell, a network utility (T1.4), any of which is a way out of the box for a workload that was never meant to hold it.

5.1 The right to launch

Execution could be policed at runtime: a guard in the path of every launch, asked each time whether to allow it. The design does not. It reaches instead for absence (construction-by-absence), the same limb the filesystem leans on. The right to launch a program is stripped from everything in the workload's world and handed back only to the paths the policy names.

A workload that tries to run a program it was not granted does not meet a monitor that weighs the request and refuses. It meets a program it cannot start, because the right to start it was never in its world. The decision was made when the world was built, not when the launch was attempted, and it costs nothing to hold.

5.2 The contract

The contract is default-deny. An empty policy launches nothing; every program the workload needs to run is named, and only the named programs run.

This closes the ordinary escalation routes without a word about them. A privilege-escalation helper, an unrestricted shell, a tool for reaching the network: none is

named in a workload's policy, so none runs, not because the framework recognises them as dangerous but because it grants nothing it was not asked to. The design keeps no list of forbidden programs. Under default-deny a denylist would be reassurance theatre: a roster of known-bad binaries, maintained forever, pretending to hold a line that the absence of a grant already holds. A program not named does not run, whatever it is.

The boundary does not depend on how a program is named, either. An absolute path to an unnamed program fails exactly as a bare name does, because what is enforced is the right to launch, not the way the launch is spelled. A policy may set where the workload looks for its programs, but that is a convenience (it turns a missing tool into a clear answer rather than a silent failure) and never the enforcement.

There is one way out, and it is deliberate. An operator can invert the posture, granting the right to launch anything the workload can read. This drops the execution boundary, and the design treats it as the boundary-dropping move it is: warned at build time and visible in the diff, never a quiet default (`footgun-warn-dont-forbid`).

5.3 The launch, not what runs after

The boundary governs the launching of a new program. It does not govern what a program does with code it reads once it is running, and that line is where the launch's concern ends and the other resources' begin.

An interpreter makes the line plain. A workload granted the right to run an interpreter may run any script that interpreter can read, because a script is data: it is read, not launched. The framework does not try to sit between the interpreter and its scripts, vetting each one. To do that it would have to judge what a given script intends, and intent is the thing the design refuses to judge (`good-boy`). The grant says the interpreter may run; what it then does is answered elsewhere: by the files it can read, the destinations it can reach, the services it can call. Execution settles what may launch. The other resource classes settle what the launched thing may do. This has a consequence worth stating outright rather than leaving between the lines: for the flagship case, an agent granted a shell and an interpreter, the execution allowlist does little of the confining work. A shell and a Python are each an open-ended executor of whatever they can read, so once they are on the allowlist the exec grant has largely spent itself, and the filesystem, network, and local-service classes carry nearly the whole of the containment. The allowlist still earns its place by keeping a compiled escalation binary off the menu, but the reader who expects it to be the primary control for an interpreted workload has the weight in the wrong class.

5.4 The non-escalation guarantee

The same line runs through a program and its libraries. A dynamically linked program needs the right to launch, and so does the loader that starts it; both are launches. But the libraries the loader then pulls in are not launched: they are read into the running program's own memory, which a read grant already covers. A library loads on the right to read it, not the right to run it. Governing that as execution would mean watching what a running program loads into itself, which is the interpreter problem again under another name.

5.4 The non-escalation guarantee

Underneath the allowlist sit guarantees the allowlist cannot weaken, and they all say one thing: a launch is not a way to gain authority the workload did not already have.

On most systems, launching certain programs is itself a privilege transition: the program runs with the authority of whoever owns it, not whoever ran it, which is how an unprivileged process has long reached for more (T3.1). The design closes this from both sides. A program that would gain authority on launch is refused the right to run at all, even if a policy names it; the floor overrides the grant, the way the filesystem's categorical denials do. And should such a program run regardless, the launch is sealed against the transition: no program a workload starts comes away with more authority than the workload had. The identity stays put, and the authority with it (*split-the-uid*). Two locks on one door: the privilege-gaining program cannot launch, and the launch cannot grant privilege.

The sharpest of the guarantees is the simplest. The right to write a path and the right to launch a path are never held over the same path. Without that, a workload allowed to write its working directory and to run a compiler could write itself a program and run it, minting its own entry on the allowlist and stepping around the whole contract. So the two sets are kept disjoint by construction: nothing the workload can write is anything it can run, and a binary it compiles or drops into a directory it controls is, in its world, an inert file.

These hold beneath the policy, and beneath the escape hatch too. An operator who inverts the posture to allow-anything-readable widens what may launch; the operator does not thereby grant a path to privilege. A workload with every readable file launchable still cannot run a program that escalates, and still cannot raise its authority by launching one. The boundary an operator can drop is the allowlist. The non-escalation guarantee is not theirs to drop (*reference-monitor*).

Execution, like the filesystem, is mostly the first limb: the right to launch is absent until the policy grants it, and granted narrowly. What execution adds to absence is a floor: that no launch, named or not, allowed or inverted, is a step up in authority.

The workload runs the programs it was given, as the unprivileged thing it already was, and the programs it was not given are not there to run.

6. Network egress

The network is how a workload reaches the world, and how the world reaches back. Give a workload the host's network untouched and it has all of it: it can bind ports, open a connection to anywhere on the internet, sweep the local subnet, and reach the private services the host runs only for itself. A workload that has been turned reaches for exactly these: a channel out to a controller, a route for the user's data to leave by, a path sideways into the home network and the services on it (T1.6).

The network is also the one resource a working agent cannot simply be denied. It has to reach a package registry, an API, a source it was told to build from. So the network is the first place both limbs are needed at once: absence, to take away the network the workload should never have, and interposition, to meter the narrow slice it needs.

6.1 Absence, then interposition

Absence comes first, and it does most of the work. In the workload's world the host's network is not there: no interfaces, no routes, no inherited connections, none of the private services the host runs only for itself. A workload mapping its surroundings for an administrative port, or for a service the host meant only for its own use, finds nothing there to map (T1.6); the lateral-movement surface is not defended, it is absent. Most of what a network boundary has to stop never has to be stopped here, because there is nothing to connect to and no route by which to try. The forbidden connection cannot be formed.

What absence cannot do is give the workload its registry back. The workload needs a route out, and a route out is exactly what was removed. So the egress that remains, the connection to the outside the workload is allowed to make, is carried by the second limb. The empty network has no path to the internet; what it has is the monitor. Each outbound connection is a request the workload makes and the monitor answers (*interpose-as-transaction*): the workload names a destination, the monitor checks it against the policy, and only if it passes does the monitor open the connection and hand it back. A connection the policy does not allow is one

the monitor never opens, and the workload cannot reach past it to dial for itself, because it has no route except through the request.

The workload names its destination; it never resolves it. It has no resolver to reach and no address of its own to offer: it asks for a name, and the monitor resolves that name and pins the address it resolved to before dialing. This is what closes off rebinding: a forged answer cannot swing an approved name onto a forbidden address, because the address the monitor dials is the address the monitor checked, and the workload was never given the chance to supply one. The decision falls on the destination that will be connected to, not the one that was asked for.

And the monitor decides without carrying. Once a connection is approved the bytes flow directly between the workload and the destination, the monitor out of the path entirely (*control-not-data-plane*). It is the point every connection is decided at, and no part of what any connection moves. That is what keeps it small enough to trust: a thing that ruled on a million bytes would have to be trusted with a million bytes; a thing that rules only on whether a connection may open has almost nothing in it to get wrong.

6.2 The contract

The contract is a spectrum, anchored at the safe end. A workload is given no network at all until the policy asks for one, and asked for conservatively unless the operator widens it: *deny-by-default* applied to reach, the same fail-safe defaults the filesystem and execution boundaries rest on [SaltzerSchroeder].

At the closed end is the empty network with nothing added back: no connection, no bind, no surface. It is where untrusted analysis and post-install scripts belong, things that have no business touching the network at all. A step out from there grants the isolated network a named set of destinations and nothing else: the workload reaches the registry and the API it was told it could reach, by asking for them, and reaches nothing else, because nothing else has been named. Wider still opens egress to the open internet, for the workload that must fetch from sources no allowlist can predict; the network is still its own, the connections are still brokered, the floor still holds. And at the open end is the host's network stack, shared directly: no separate network, no broker, the host's local services reachable once more. This last is the escape hatch, and it reinstates the very surface the chapter began by removing (T1.6), so the design will not compile it without an explicit reason written down beside it (*footgun-warn-dont-forbid*).

Beneath every posture but the empty one runs a floor no policy can lift. A handful of destinations are denied in every mode, the widest included: the address a cloud platform answers its own metadata on and the link-local ranges, the standing targets of request-forgery and infrastructure reconnaissance. Even the open-internet

6.3 What the boundary does not claim

posture cannot reach them, because the denial is not a policy choice but a property of the boundary. The broker holds one more line of the same kind: it will not become an open relay. A workload cannot hand it an arbitrary internal address and have it dialed, not a sibling's network, not the host's own services. Reaching a host service is its own named grant, deliberate and visible, never something egress backs into.

6.3 What the boundary does not claim

The boundary governs where a workload connects. It does not govern what the workload says once connected, and the line between those two is not a corner the design cut: it is the edge of what a boundary of this kind can enforce at all.

That a workload never opens a connection the policy forbids is a safety property: the forbidden connection is a thing that, once it happens, cannot be called back, and the monitor's whole purpose is to see that it does not happen. Complete mediation of the channels a workload may open is enforceable, and the monitor enforces it. What travels over a channel that was allowed to open is a different kind of thing. It is information flow, and information flow is not a safety property: no monitor watching one execution at a time can enforce it [Schneider]. So the monitor does not look. There is no payload inspection and no interception of encrypted streams; the bytes over an approved connection are opaque to it by design.

This draws the residual exactly. When an operator allows a workload to reach an external API, what the workload chooses to send there is a data-governance question, not a confinement one (T1.8). The boundary mediates the channel completely and makes no claim about the payload. The claim it does not make is one no boundary of this shape could honour, which is why stating it plainly is the honest thing rather than a confession. The host-mode residual is of the same character: an operator who shares the host's stack has reopened the lateral surface on purpose (T1.6), and the boundary records the choice rather than overriding it.

Across all of it the two limbs divide the surface the way the third chapter said they would. Absence takes the network the workload should never have had and leaves nothing to mediate; interposition meters the one slice that could not be removed, and meters it at the point of decision without ever touching the flow. Together they enforce a clean safety property (no forbidden connection occurs) and they stop exactly where safety stops, at the payload, which was never theirs to police.

7. Local services

A workstation's most sensitive services do not listen on the network. They listen locally, on sockets in the user's own session: the SSH and GPG agents holding the keys, the display server, the audio server, the session bus, the container daemon. These are the conveniences a desktop is built from, and a workload running as the user can reach every one of them.

For almost all of them, reaching is enough. Connection is authorization. A workload that opens the SSH agent's socket can have the user's challenges signed and authenticate as the user to every host that trusts the key (T3.7); one that reaches the display server can read the clipboard and watch the keystrokes (T2.6); the Docker socket is, for practical purposes, root on the machine. There is no second step where the service asks whether this particular client was meant to have it. The socket is the authority, and under the default-uid model every one of the user's sockets is open to anything running as the user. The workload inherits the user's whole local session.

So the boundary cannot sit inside the conversation, asking whether a given request to a service is allowed; by the time there is a conversation, the authority is already handed over. The boundary has to sit at the connection, and then, for the services where even an allowed connection hands over too much, reach past it. What follows is that progression: first making the connection a mediated thing at all, then the graded set of answers for connections that cannot simply be granted.

7.1 The paths, then the connection

The obvious way to confine local sockets is to keep doing what the filesystem chapter did: build the workload a view with only the allowed sockets in it, and let the absent ones be absent. It is the wrong tool here, and the reason is worth stating, because it shapes what follows.

A socket is not data at rest. A socket path that appears in the workload's world is not a fact it can read: it is an authority it can exercise, out of band, the moment it sees it. So a view that contains the allowed sockets still leaks: the workload reads its

7.2 The contract

own grants off what is present, and a present socket is a connectable one whether or not the monitor was watching. The path model is brittle besides: services that come and go through a session, sockets the host addresses through rules the policy would have to mirror exactly, and names the system resolves to a live endpoint though they were never files (an abstract socket on one platform, a reserved DOS device name on another), neither of which a path scheme can name at all. Each is a seam. Together they say the filesystem is the wrong layer to hold this boundary.

So absence is applied to the paths wholesale. No host socket path is present in the workload's world: not the forbidden ones, not the allowed ones, none. There is nothing in the view to enumerate, probe, or connect to behind the monitor's back, and where a name could reach an endpoint without being a file the workload still has no way to address it. The local surface is empty, the same way the network was.

What the workload has instead of paths is the monitor. A workload that needs an allowed service asks for it by the name the policy gives it, and the monitor, if the policy allows it, opens the real connection on the host side and hands the connected channel back (*interpose-as-transaction*). The application connects the way it always has; the call becomes a request, and the request is answered or refused. Enforcement moves off the filesystem and onto the connection, where it can be checked the instant it is made and audited every time, and where a service that restarted or moved since the workload began is reconnected correctly, because the name is resolved fresh each time rather than pinned to a path that may have gone stale.

7.2 The contract

The contract is default-deny, and the default holds by construction rather than by rule: a service the policy does not name cannot be named in a request, because the workload has no path to it and no handle on it: only the set of names the policy gave it. There is nothing to add a denylist to. The dangerous services (Docker, the system bus, the legacy X11 server) are not refused; they are simply never named, and a service that is never named is one the workload cannot ask for.

Asking by name rather than by path buys more than tidiness. The name is the workload's only handle on the service: it has no path, no address, nothing but the word the policy gave it, and so what stands behind that word is the monitor's to choose. To the operator inspecting a kennel before it runs, the choice is laid bare: the names the workload will ask for, and what each one really reaches, side by side; the operator sees the topology the workload never does. To the workload there is only the name. Everything that follows is a different answer to one question: when the workload asks for a service by name, what does the monitor connect it to?

7.3 The raw channel

The simplest answer is the real service, connected raw. The monitor opens the connection and hands over the channel, and from there it does not listen: it granted the connection, it does not police the protocol that runs across it (*mediate-use-not-reach*). A workload granted the audio server can do anything PipeWire allows (play, and also capture) and the monitor reads none of it.

This is the same edge the network boundary has, in the same place. That a connection the policy forbids never opens is a safety property, and the monitor enforces it completely: the forbidden connection is a thing that, once it happens, cannot be called back, and the monitor sees that it does not happen. What is said over a connection the policy allowed is a different kind of thing (the use of the channel rather than the reach to it), and a monitor watching one exchange at a time cannot bound it [Schneider]. So a raw grant is a raw grant of whatever authority the protocol confers: no more than was asked for, but no less.

Where the protocol's whole authority is what the operator meant to grant, that is the right answer and the end of it. The audio server is audio; granting it grants playing and capturing sound, which is what audio access is. The answers that follow are for the services where the protocol's whole authority is far more than anyone meant to hand across in one piece.

7.4 The facade

The session bus is the clearest of those. A connection to D-Bus is a connection to everything on it: the notification service, the secret store, the network settings, the power controls: the desktop's whole cast of services, reachable through a single socket. Handed the bus raw, a workload has them all; connection is authorization, and here the authorization is the session entire.

But the bus is not an opaque stream the way a signing agent's socket is. It is structured (what travels on it is messages, calls to named services and named methods), and structure is something a boundary can hold. So where the bus cannot be handed over whole, it is not handed over raw.

What the workload is given instead is a facade: something inside its own kennel that wears the shape it expects (a socket that looks and answers like the bus it meant to reach), so the application speaks to it unaltered. The facade decides nothing. Its whole job is to present the right face and carry what the workload sends across the kennel's IPC surface to the other side, where the deciding happens: a separate service kennel, or the host context, a place the workload has no handle on. There the messages are parsed and policed, and only the calls the policy named

7.5 The private instance

go on to the real bus. A workload granted the notification service can post a notification; the message crosses to an enforcer it cannot reach back into, and every call it was not granted stops there.

This is the shape of every brokered service, thrown into relief by the danger of this one. The side facing the workload is only ever presentation; the side that decides is always across the boundary, beyond the workload's touch, which is what makes the deciding trustworthy, because an enforcer the workload cannot reach is one it cannot turn. It is also what keeps the trusted core small. Policing D-Bus means understanding D-Bus, and understanding a protocol is a great deal of code, exactly what a reference monitor is meant not to accumulate. So it accumulates nowhere near the monitor: the protocol logic lives in a confined service of its own, on the far side of the IPC surface from the workload it guards against, and the monitor brokers the connection and reads none of what crosses it (`control-not-data-plane`). The workload faces a mask, and the work goes on somewhere it cannot follow.

7.5 The private instance

Some services cannot be filtered into safety, because there is no clean subset of the protocol to allow; and they cannot be handed over raw, because raw is catastrophic; yet the workload has a real need the service answers. The display is the standard case. A workload may legitimately need to put a window on the screen; a connection to the user's display server lets it read the clipboard, log every keystroke, and capture the screen of every other window (T2.6). The protocol is not a handful of discrete privileged calls to allowlist one of; it is a continuous compositing relationship, with no message-level line through it that leaves something both safe and useful.

So the design neither filters nor refuses. It substitutes. The workload is given a display server of its own (a private, nested instance) and composes into that. It draws its windows, receives the input meant for it, and behaves exactly as it would against the real one, because as far as it can tell it is talking to an ordinary display server. What it cannot do is reach another window's input or output, because in its own server there are no other windows. The user's real display is never connected; the private instance stands on the far side of the boundary, in its own confinement like every brokered service before it, so the blast radius of a compromise is that instance and nothing past it.

The private instance is the answer; a raw connection to the real display is the residual the answer exists to avoid, and it is worth naming, because an operator can still ask for it. On Wayland, built with client separation in mind, a raw connection to the real compositor still carries the clipboard and the screen until a message-filtering facade for it lands, so where a private instance is not in place that grant is a real

grant of that reach. On X11 it is past a residual. X11 has no notion of one client kept from another; any client reads any other's input and screen as a matter of protocol. There is nothing to filter and nothing to contain, and a workload handed the user's real X server is handed the whole session's eyes and ears. Granting it is equivalent to turning the boundary off, and a workload that needs it is operating outside the model.

7.6 What is not brokered

The last answer is no answer at all: the service is not brokered into the kennel by any of the three routes. It is right for exactly one kind of service: the one where there is no safe subset to filter, no useful private instance to substitute, and raw access is the user's identity itself.

The SSH and GPG agents are that kind. A socket to a signing agent is the standing authority to authenticate as the user: to sign whatever challenge is put to it, with no attestation of what is being signed or to whom, no second check that the asker was meant to ask, and in some protocols a path to the key material behind it. There is no safe subset of "sign this" the framework could *mediate*: a facade that parsed the agent protocol and passed selected requests would be vouching that a given signature is a legitimate one, which is a claim it has no ground to make, and bounding which key the agent uses does not bound what that signature then authenticates. And there is no useful private instance, because the value of the agent is precisely the user's real keys: a private agent holding none is just an absent one.

So the framework builds no signing facade: it will not stand a mediated service in front of an agent and vouch for what crosses it. This is a bound on what the framework constructs, not on what the operator may reach. The raw channel of §7.3 remains available for an agent socket as for any other, and an operator who writes that grant has connected the workload to the agent with the framework parsing nothing and vouching for nothing; it is a warned footgun the operator owns, loud and tagged, not a sanctioned path. What the framework refuses is to *be* the thing that mediates the signing, never to let the operator express a raw connect. The legitimate need behind the SSH agent, a workload that must reach a host over SSH, is real, and it is met a better way than the raw socket: a credential bound to a single destination, with the real key kept on the host side and out of the kennel entirely. That is a boundary of its own shape, standing on the network rather than the socket, and it is the subject of the next chapter. The GPG case has no such re-originated form, for the reason the next chapter draws out: it is attestation, and attestation does not belong in confinement at all.

The two limbs meet here the way they met at the network, and then interposition goes further than it did there. Absence takes the paths, so there is no service to

reach except by asking. Interposition answers the asking, and because the workload's only handle is a name, the monitor is free in what it puts behind that name: the real service raw, a facade that filters it, a private instance that stands in for it, or nothing at all. Connection was the authorization, so the design made the connection the transaction, and then made the transaction answerable in whatever measure the service's danger demands. Each rung leaves a residual, named where it falls (the protocol's full authority where it is granted raw, the correctness of the enforcer behind a facade where it is filtered, the blast radius of the private instance where one stands in), and the service whose only honest answer was refusal is handed to the chapter that can give it one.

8. Cryptographic services

A workload that does real work reaches for the user's cryptographic agents: it pushes to a remote over SSH, signs a commit, authenticates to a host it was told to reach. These are among the most dangerous sockets in the session (a connection to a signing agent is the standing authority to act as the user), and like every other resource they are default-deny: no cryptographic authority reaches a kennel unless the policy grants it, and the agent socket itself never crosses at all.

Whether a grant can exist, and what shape it takes, turns on a line drawn sharply here but not invented: it is a standing law of the whole design (authentication-never-attestation). A cryptographic agent does two different things under one socket. It authenticates: proves, now, to one party, that the user holds a key. And it attests: asserts, durably and portably, that the user vouches for these bytes. The boundary can carry the first and must never carry the second, and the reason reaches far past signing.

A capability the monitor mediates may be authentication-shaped: a constrained, host-verifiable act it can bind to a property it is able to check (may I reach this host, may I open this connection, render this frame, carry this transport). That is what the interposition limb has been carrying all along; every facade of the last two chapters was an authentication-shaped capability bound to a checkable edge. A capability may never be attestation-shaped: vouching, signing, issuing a secret ("trust that this is so"), whose entire worth comes from the trust of its origin. And a kennel's origin is untrusted by construction. A kennel that vouches is a trust root placed inside the boundary built to confine it: a trust claim with nothing behind it. You cannot automate trust for an untrusted thing. SSH and GPG are the two services the user runs that fall on opposite sides of this one law.

8.1 Authentication

SSH is the authentication-shaped case (the destination is exactly the checkable property the capability binds to), and it is solvable, though not by brokering the agent. The agent's protocol signs a blob that carries no destination, so an agent, or

any broker filtering in front of one, can bound which key signs but never which host the signature authenticates to. A workload holding the socket opens it directly, asks for a signature over a challenge it built for a host of its own choosing, and authenticates as the user anywhere that key is accepted: a host the policy never named, reached by reusing a key across destinations the operator never meant to join (T1.6). The agent is a destination-blind oracle, and no broker can make it destination-aware, because the destination is not in the protocol to filter on.

So the design changes what the workload holds. The signing authority stays out of the kennel; in its place the workload is given a synthetic credential, and here the direction is easy to get backwards and worth stating flat. For each destination granted, the trusted side mints a fresh keypair. The kennel receives the *private* key; the trusted side keeps the *public* key and binds it to the real destination. The instinct that the private key is the precious secret is the one to drop: this synthetic private key authenticates to nothing real on its own: it is a disposable index. The security-bearing half is the public key, because its binding to a destination is what the workload must never reach. The workload holds the index; the trusted side holds the table, and the destination follows from which credential authenticated, settled when it was minted, never read from anything the workload asks (*split-the-uid*).

What the trusted side does with a matched credential is re-originate. It makes a new connection outward, as the operator, with the operator's own real key (the key that never entered the kennel and never will), carrying the workload's command through to the bound destination and signing against that host and no other. The workload reaches the re-origination point over the egress machinery of the previous chapter, allowlisting only that point and never the destinations behind it. So cross-host reuse, the whole danger of the oracle, is not guarded against but made impossible to express: the key that could go anywhere is never in the kennel, and the key that is there leads to exactly one host by construction. The re-origination point is no bespoke proxy in the trusted path: it is the same ubiquitous SSH server software every platform runs, configured to delegate one decision to the trusted authority, so the only new trusted code is the small resolution from credential to destination.

8.2 Attestation

GPG is the attestation-shaped case, and it is not solvable, because there is nothing to engineer: only a contradiction. A signed commit or release tells readers who will never see the kennel that the user stood behind these bytes; its worth is the user's trust, and the user's trust is the one thing the boundary exists to withhold from what runs inside. To let the kennel attest as the user is to mint trust-bearing claims from inside a boundary built to deny trust. No broker resolves it: a broker would have to decide which attestations the user *means*, which is judging intent,

which the design refuses (good-boy). So the framework builds no GPG facade, and unlike SSH there is no re-originated form that would save it: the need itself does not belong in confinement. What is refused is the mediated service, the framework standing behind a signing act as safe; the raw agent socket, like any raw grant, an operator may still force as a warned footgun they own, and the runtime does not pretend the connect is unavailable. The framework will not *build* the attesting broker; it does not police what the operator connects by hand.

And the refusal is a category, not a case. No facade the monitor ever grows (none built today, none added later) may be a secrets broker or a signing service, for the same reason. Handing the act off to a keyring, a hardware token, a vault does not rescue it; it only relocates the attestation to “I am authorised to fetch this on the workload’s behalf,” which is the broker vouching for an authority it cannot be trusted to vouch for. The whole category is refused, not merely left unbuilt.

What a kennel legitimately needs by way of trust material comes to it the other way around: not brokered at runtime but placed at construction. A credential a workload truly requires arrives as a signed construction parameter, present in the kennel because the operator put it there, declared and visible in the policy (Chapter 4). That is the authentication-shaped form of the same need: the kennel has what it was given, rather than asking a peer to vouch for it at runtime. The construction limb supplies trust; the interposition limb never manufactures it.

So the two services divide on the one law, and the law divides more than them. Authentication is a proof to one party, bound to a property the host can check and re-originated with the real key kept home. Attestation is a claim to everyone, bound to nothing a host can verify, and not to be made from inside a boundary built to withhold the very trust it would carry. That check-ability is what decides whether the monitor may *mediate* a capability or must confine itself to *connecting* the workload to it and stepping out. To mediate is to parse the protocol and pass the requests that clear a rule, which is the monitor vouching that each passed request is a legitimate one; it can only do that honestly where the property it vouches for is one it can check. Authentication has such a property, the destination, so the monitor mediates it and binds each act to the host it is for. Attestation has none, so a facade that mediated it would be vouching for an authority it cannot verify, and the monitor refuses to build one. What the operator connects raw is a different thing: a bare fd the monitor parsed nothing on and vouched nothing for, the operator’s to own. The line is not signing versus not-signing but mediate versus connect, and it falls where check-ability does. Every facade the monitor offers stands on the first side; the entire category of vouching stands on the second, never mediated, and where the operator forces a raw connect to it the monitor carries the bytes without standing behind them. The key that goes one place is all the workload is given, and the standing to vouch for the user is given to no workload, by no facade, ever.

8.3 The man-in-the-middle we can't be

What is refused is the standing to vouch *as the operator*. The workload vouching *as itself* is a different act entirely, and nothing forbids it. An operator who wants a kennel to sign draws a second identity, a signing key distinct from their own, and hands the workload its private half the way any other trust material enters, placed at construction and declared in the policy. The workload then signs in its own name, and the signature is worth exactly the trust that identity has earned, which is the operator's to confer and the operator's to bound. No facade mediates it and no agent is exposed, because the workload holds the key rather than asking a broker to wield the operator's; the construction limb supplied the trust and the interposition limb never entered. The line the law draws runs between the two identities, not across signing itself: the operator may give a kennel a voice, on the condition that it is the kennel's own and never a theft of theirs.

8.3 The man-in-the-middle we can't be

There is a move the SSH bastion seems to license. If an authentication can be re-originated, why not a request: let the workload reach a remote API without holding the token, and have the framework insert the credential into the workload's outbound HTTPS on the way past. The workload never sees the secret; the same shape that made re-origination clean, applied to the web. The goal is right, which is what makes it tempting.

It fails on direction. Re-origination proxies between two connections the bastion is an endpoint of: it terminates one and originates another, both real, wielding a key that was the operator's to begin with and binding it to a destination the host can check. Header injection is not that. The framework is not an endpoint of the workload's HTTPS session, so to add a header it must terminate the transport under a certificate authority the workload is forced to trust and re-encrypt to the origin. That CA is the whole problem. The SSH bastion's trust anchor was scoped to one thing we owned, the synthetic key we minted; a CA the workload trusts is unscoped by construction, able to present a fabricated identity for `google.com`, for a package registry, for any origin the workload will ever reach. We would not be vouching to the remote that the workload is authorised; we would be vouching to the workload that every remote is who it claims to be, having made ourselves the issuer that says so. That is a false attestation in the direction §8.2 refused, aimed at the confined side: TLS's entire authentication guarantee inside the kennel would root in a CA we hold, so a single flaw in that proxy impersonates the whole internet to the code we are supposed to be confining. We would have fixed one credential handoff by making the framework a trusted issuer for every remote at once. So it is not built. The credential a workload needs enters at construction, held by the workload as its own; the destination is mediated as reach, and the encrypted stream is left to the two parties that own it.

9. OCI images

Developers package their work as OCI images, and an agent or a build is often handed one as the environment it must run in: a base image with a toolchain, a service's published container, a dependency that ships no other way. A confinement tool that cannot run them is a tool that does not get used, whatever it thinks of the format. So Kennel runs OCI images. What follows is how it does that without trusting them, and without taking on the execution model they arrive with, because both the image's contents and the machinery that normally runs it are incompatible with the boundary.

9.1 Not the engine

An OCI image is normally run by Docker, and Docker is the first thing that cannot cross into this design. Its daemon is a root-equivalent orchestration service: the verb that creates a container is, in effect, the authority to become root on the host. There are two ways to let a confined workload reach it, and both give the boundary away. Proxying the Docker socket puts its entire protocol (a large, stateful parser) into the trusted core, in front of a verb that is host-root. Running a container engine inside the kennel instead (Docker, or a rootless one like Podman) hands the workload the two kernel features most often used to escape confinement, the very ones the boundary exists to withhold. The easy bridges are the ones that undo the design, so it builds neither.

9.2 An image is a filesystem

The reframe that makes the rest possible is to stop treating an image as a thing to be run and treat it as what it physically is: a filesystem. An OCI image is layers of files and a little configuration describing how to start one of them. Booted directly as the root of a kennel (under the same boundary every other workload runs under, with no engine in the loop), it is inert content, and inert content is what the filesystem chapter already knows how to confine (*construction-by-absence*). The container does not need Docker; it needs to be the world a confined workload runs inside,

and that world is built the same way every other kennel's world is built. The whole apparatus of the engine collapses into a problem the design had already solved.

What the image brings that an ordinary kennel does not is that the world itself is now untrusted. In the filesystem chapter the constructed world was assembled from the host's own files and the operator's own directories; here the bulk of it arrived from a registry, authored by someone the operator has never met. That changes what the design must promise, but not how it confines: the image is substrate, declared and pinned, and the boundary around it is the boundary around any kennel.

9.3 Ingesting without trusting

An image arrives wrapped in parsers: the registry protocol that fetches it, the manifest that indexes it, the compressed archives that are its layers, and the runtime configuration buried inside it. Every one of those is code reading untrusted input, and a confinement tool that parsed them in its trusted core would have given back at the front door everything it defends at the back. So none of them run there. They run where everything untrusted runs in this design: inside a kennel.

Fetching and unpacking the image is itself a confined workload. A builder kennel runs the ordinary user-space image tools to pull the image and lay out its files, and the broad network egress a registry pull requires (the widest reach anything in the flow needs) lives only there, fenced inside that one kennel under a fetch policy Kennel ships and signs itself. The operator never authors the broad-egress step and never signs it; they request a fetch, and the vetted policy that knows how to reach a registry safely is supplied for them (`request-dont-author`), overridable only by a key trusted to widen it. A bug in any of those parsers is a bug in a confined, unprivileged process, contained exactly as any other workload is. The trusted core gains no registry client, no manifest reader, no archive extractor.

The image's own startup configuration is handled the same way, at the other end. An image says how it wants to start (an entrypoint, a default command, an environment), and rather than parse that in the trusted core or make the operator transcribe it by hand, a small Kennel-shipped launcher reads it from inside the running kennel, at the workload's own authority, and starts the entrypoint. One part of that configuration cannot be passed through as the image wrote it. A handful of environment variables are injection vectors that a loader or a language runtime acts on the instant a program starts; passed unfiltered from an untrusted image, they are a free hand into the workload's own startup. So the launcher strips those names before it applies the rest: absence at finer grain, the same move the filesystem chapter made on a secret-bearing file, turned on the environment. It is the stripping the

system itself performs when a program crosses a trust boundary, reproduced here because the image's environment is exactly such a crossing.

9.4 What the operator takes on

Running someone else's whole filesystem is a trust decision, and the design refuses to make it quietly or to make it for the operator. Declaring an image as the substrate is a loud grant, in the same class as sharing the host's network (`footgun-warn-dont-forbid`): it carries a required reason, and the compiler derives a substrate-trust exposure from its presence and surfaces it among the kernel's risks (T3.8). The operator is told, in the policy and in the risk report, exactly what they are taking on.

And what the design takes on in return is bounded, deliberately. Its posture over an image is confinement, not content integrity. It pins the image by digest (a tag is resolved to a digest when the image is fetched and frozen there, so what runs is exactly what was pinned) and it confines what the image can do to the same degree it confines any workload. It does not vet the image, launder its contents, or assert anything about the bytes it did not write. The provenance is the digest and the operator's declaration, and nothing more is claimed; the signature pins the substrate, not each binary inside it, because inside an image there is no per-file pin to make. This is the same honesty the network chapter kept about payloads: the boundary governs what the thing may do, and is silent on what the thing is.

That honesty extends to the seams the design knows about. Ingesting an image safely means ingesting it without privilege, and an unprivileged unpack cannot preserve the image's own file ownership, so the unpacked tree comes out owned by the operator, which hands the workload write access where a foreign-owned file would have refused it. The design re-imposes the boundary that flattening erased, holding the image's executable surface read-only beneath the workload; but it does so from the image's declared shape rather than a walk of every file, so it is honest that the re-imposition has edges: an image that drops privilege in its own startup, or installs its code outside the usual system directories, is locked less completely than one that does not. Stronger integrity than the confinement floor (content-addressing the layers, verifying them at use) is available as opt-in hardening, each rung surfaced as a risk it retires, above a floor that is confinement and says so.

Across all of it the design declines to make an exception. A foreign format arrived with its own execution model and its own assumptions about what may be trusted, and neither was allowed in. The engine dissolved into a filesystem the boundary already knew how to hold; the parsers that came with the image were made to run inside the boundary rather than in front of it; and the trust the format asks the

9.4 What the operator takes on

operator to extend was handed to the operator to extend, loudly, with the design confining what it could not vouch for and saying plainly where its vouching stops.

10. Delegation

Delegation here does not mean one person handing work to another. It means the framework handing a workload one of its own powers (the power to bring a kennel into being) so that a confined, untrusted workload can cause a second kennel to exist. Instantiating a kennel is normally the operator's act; here that act is lent to a workload the operator has explicitly granted it. And what the workload brings into being is not a child it owns but a sibling it cannot reach into: kennels have no hierarchy, and a spawned kennel is no more the spawner's than any other on the machine.

Why lend a workload such a power at all is the lethal trifecta, and answering that is what follows.

10.1 The trifecta

An agent earns its keep only when it can both run code and reach the network: read the repository and fetch the dependency, run the test and call the API. But a single workload that can do both is the lethal trifecta in one box: code that can be steered by a prompt injection (an agent is exactly that, T3.7), private data within its reach, and a path out to carry that data away. Confining such a workload tightly does not dissolve the danger, because the danger is not what it might escape to but what it was openly granted: both halves, in one place, under one compromise.

The industry's answers are a full virtual machine for each task (Firecracker and the like), which is heavy and elaborate to orchestrate, or a container engine nested inside another (Docker-in-Docker), which is root-equivalent and leaks. Kennel already holds a lighter primitive. A workload can be given neither half (no code execution it can aim at the network, no network it can hand code) and still get its work done, by asking for a sibling kennel that holds exactly one half and speaking to it through a channel. The agent runs no untrusted code and reaches no network; the tool it spawns runs the code with no network, or reaches the network with no code to run. Each kennel holds one leg of the trifecta. The dangerous combination is assembled in no single kennel, because the thing that would assemble it holds

none of it. That is the claim in its exact form, and it is worth stating precisely rather than as a slogan, because it holds per kennel and not per session: an agent that brokers between two complementary siblings can still route data from one to the other across the channel it holds to each, reassembling the trifecta with an extra hop in it. The residual is real and named later in this chapter; the headline is that no one kennel is ever handed the whole, not that the whole can never be composed by a workload wiring siblings it was granted.

10.2 Request, don't author

The rule that makes delegation safe is that the workload never writes policy. It cannot author a grant, invent a capability, or compose a kennel of its own design. What it can do is name a template the operator has already signed and installed, and fill in the few fields that template marks as fillable. Everything else in the template is frozen and inherited exactly as signed (`request-dont-author`).

This is why lending the power to instantiate does not lend the power to grant. The capability floor of every spawn is the signed template's, and nothing the workload does raises it: a template that fixes its network off stays off, whatever the workload writes, because the only writes that take are the ones the template opened, each within the bound the template set. The operator consented once, when they signed the template into the store; every spawn from it is a reference to that consent, not a new one. So the entire surface a workload controls is the handful of blanks the template left for it: not the policy, only the values in fields the policy already fenced.

A workload should not have to discover that surface by trial, firing speculative requests to learn what the framework will accept. The grant is open to inspection: a workload can ask the framework for its own delegation grant and be told which templates it may instantiate, which fields of each it may fill and within what bounds, and how many siblings it may have at once. It learns the shape of the fenced blanks before it fills them: the reading side of requesting rather than authoring. And the grant is loud: holding the power to instantiate siblings is an exposure the operator declares deliberately and sees surfaced among the kennel's risks (T3.9).

10.3 Siblings, not children

What the workload brings into being, it does not command. The framework is the spawner; the requesting workload and the spawned kennel are siblings, joined by the channel between them and nothing else. The requester cannot trace the sibling's execution, send it signals, or reach into its memory, and that absence is not an oversight but the whole point. A spawn exists to create an isolation that did not exist before; if the spawner could reach into what it spawned, that isolation would

be defeated at the instant of its creation. So the spawned kennel runs under its own boundary, as separate from its requester as from any other kennel on the machine.

The framework that does the spawning stays where it always does: out of the byte path. It checks the grant, fills the template's blanks with the workload's values, builds the sibling, and hands back the ends of a channel; it does not sit in the conversation that follows. When the sibling is an agent's tool (a tool server speaking the Model Context Protocol over that channel), the protocol is the two kennels' business, parsed by neither the framework nor anything in its trusted core (*control-not-data-plane*). And the power to delegate does not spread on its own: a spawned kennel can spawn further siblings only if its own template grants it that, so a single delegation does not open an unbounded chain, and a ceiling on concurrent siblings bounds the rest.

10.4 The residual

What delegation cannot do is make the workload's two controlled inputs safe on the operator's behalf. The first is the values it writes into the template's open fields. A field the operator left too wide (a destination pool that admits more than it should, a pattern that lets a path slip through) is a hole the operator signed, and no machinery downstream closes it, because the template said those values were the workload's to choose. The bound is only as good as the operator drew it.

The second is sharper, because it is the trifecta returning by the back door. An agent granted the power to spawn a tool that reaches the network, and also a tool that touches the filesystem, holds no dangerous combination in any single kennel, but it holds a channel to each, and it can wire those two channels together itself. The leg that reads private data and the leg that reaches the network are reunited not inside a kennel but in the agent that brokers between them. The decomposition holds per kennel and can be undone across kennels, by the one party with a channel to both. So the grant that matters is not only which templates a workload may spawn but which it may spawn together: complementary halves within one workload's reach are the whole trifecta with an extra hop in it. The design makes the assembly visible and deliberate; it does not pretend a workload handed both halves cannot put them back together.

Delegation is the framework lending a workload a power that was the operator's, and getting back the usefulness of an agent that can reach beyond itself without ever holding, directly, the danger of doing so. It is made safe by three things at once: the workload requests from signed templates rather than authoring policy, so it can instantiate only what the operator already consented to; what it instantiates is a sibling beyond its reach rather than a child under its hand, so the isolation survives its own creation; and the capabilities dangerous together are handed out apart, so

10.4 The residual

no single confinement holds the combination. What remains is the operator's to weigh (how wide the blanks, and which halves to place within one workload's reach), and the design's part is to keep that weighing honest and in view.

11. The capability mesh

By the time a kennel is running, the design has spent four chapters making sure it cannot see any other. Its filesystem holds no other kennel's files; its network reaches no other kennel's services; its local surface names no other kennel's sockets. Two kennels on the same machine are, by default, as invisible to each other as two machines (*construction-by-absence*). That is usually right, but not always. Real work sometimes needs one confined workload to use a capability another confined workload provides: an application that must render to a display server it does not itself run, reach a session bus through a broker it does not host, read from a cache that lives in a kennel of its own. The mesh is the deliberate exception that allows this: without either kennel gaining the other's authority, and without disturbing the rule that kennels have no hold on one another.

The exception is kept as narrow as the rule. A provider and a consumer remain siblings under the framework, neither nested in the other; the only thing that passes between them is the single connector the operator declared and the framework brokered. All this adds is how that one seam is opened safely between two confinements that still cannot, in any other respect, see each other.

11.1 Name, not peer

A capability enters the mesh by being offered and reached, and the two declarations are deliberately blind to each other. A provider offers a capability under a public name (the way a service announces an interface, not the way a process announces itself) and lists nothing about who may consume it. A consumer reaches for a capability by that same name, and names no provider. Nothing in either declaration points at another kennel; there is no peer reference anywhere on the policy surface. A consumer asks for the display, or the cache, not for a kennel, and the framework resolves that name, at runtime, to whoever offers it.

That blindness is what lets the policies stay independent. Because no declaration names another kennel, each policy is complete on its own (it can be written, compiled, and signed with no other kennel in view), and the cross-kennel reach it asks

for is resolved later, against whatever providers the operator has installed. The reach is real, but it is never authored as a link; it is two independent declarations that the framework, holding both, recognises as a matched pair.

And the matching is deny-by-default, the discipline carried straight from delegation. A workload cannot author a cross-kennel reach any more than it could author a policy (`request-dont-author`); both the offer and the use are signed declarations, and the framework brokers only the pairs where both sides declared and the operator signed both. What a kennel can reach is a projection of those declarations, computed fresh, never a standing list of grants left lying in the daemon. Where a public name is not enough (where more than one kennel might offer the same name and a consumer must be sure of the one it meant), provider and consumer can pair on a second key, a private one this time. It is not a secret in the sense of a credential whose strength is its concealment; it is the other half of the lookup. The public name takes a consumer to everything offered under it, the way a title takes you to a shelf; the private key picks the one intended provider from among them, the way a catalogue number picks the one volume. Both are lookup keys (one advertised, one not), and the framework requires both to match before it brokers the pair. The private key names no kennel either: it is a value the intended provider and consumer happen to share so the match lands on the right one, not a reference either holds to the other.

11.2 Resolution is runtime

A cross-kennel match cannot be settled when a policy is signed, and the design is careful not to pretend it can. Each policy is compiled and signed alone (the compiler only ever holds the one in front of it), and the providers that can answer a consumer are whatever the operator has installed by the time the consumer runs, not whatever happened to exist when the consumer was signed. To promise at signing time that a given reach will resolve would be to make a signed, immutable artefact depend on a separate, mutable, independently-authored one: a promise the system has no way to keep. So matching is a runtime property, and the design puts it there rather than faking it earlier.

What can be checked early is checked early, and only that: holding one policy, the compiler confirms each offer and each use is well-formed and that a reserved name is one the policy is entitled to claim: everything decidable from the single artefact, nothing that consults another kennel. The rest waits for construction, when the consumer's name is resolved against the catalogue of what is installed. A consumer says, of each capability it reaches, whether it can live without it. A required capability that resolves to nothing (no installed provider offers the name) fails the kennel's construction loudly, rather than starting a workload missing something it depends on; an optional one that resolves to nothing is simply skipped, and the

workload bears the absence. It is the hard-versus-soft dependency distinction system draws between what a unit requires and what it merely wants, evaluated here against what the deployment has rather than what a policy hoped for.

11.3 Typed shapes

A capability also carries its shape (the kind of transport by which it is reached), and the consumer must state the shape it expects, because the ways of reaching a capability are not interchangeable. A connected socket, a name made reachable on the bus, and a brokered channel are acquired through entirely different means; a consumer that expects one and is handed another has been misdelivered, not served. So the broker matches shape as well as name, and refuses a mismatch at the moment of connection rather than letting a consumer find it as a failure later.

The set of shapes is small and closed on purpose, and the reason is the discipline that has governed every facade so far. Each shape is a transport the framework already mediates: the local socket of the services chapter, the bus reached through that same chapter's broker, the channel that carries a delegated sibling. The mesh adds no new way to move bytes between kennels; it adds a way to declare and broker the ways that already exist. A single catalogue and a single broker describe every cross-kennel reach uniformly, and widening the mesh means teaching it another transport the framework already brokers, never a new wire protocol in the trusted path (*do-less*). The broker itself stays where every mediator in this design stays: it matches, it authorises, it mints the connector, and it steps out of the bytes that then flow (*control-not-data-plane*).

One thing the broker never does is hand a consumer a connection it has not yet asked for. A capability is reached on demand, when the workload acts, not pressed into the kennel at construction, which is also what lets a provider stay down until it is first reached and come up only then, so a standing service that no running kennel is currently consuming costs nothing to have offered.

11.4 The standing provider

What the mesh carries that delegation did not is permanence. A delegated sibling was ephemeral: spun up for a task, reaped when the requester was done. A mesh provider is usually a standing service: a confined kennel the operator enables once and leaves up, serving many consumers across its life: the confined display service the earlier chapters pointed toward, a session-bus broker, a shared cache. Permanence changes the trust the provider carries and the residual it leaves (T3.10).

A provider is, almost by definition, code that needs a foothold in the host: the display service must reach the real display, the bus broker the real bus, the cache

the real storage. The ordinary way to give code a foothold in the host is to run it in the host: unconfined, at the user's full authority, trusted because there seemed nowhere else to put it. Kennel puts it in a kennel instead. A provider is itself a confined workload, granted exactly the one leg into the host its job requires and nothing more: the same confinement-and-enablement structure every other kennel runs under, turned on the very services that would otherwise be the unconfined host-side code everything else depends on. The trust it carries is layered accordingly, by the mechanism delegation already established. A reserved name cannot be authored by a policy that simply fancies it: a reserved namespace belongs to a tier, and only that tier's key may sign a template claiming a name within it. The `org.projectkennel.*` namespace is the project maintainers', always: a name under it is honoured only on a maintainer-signed template, and no host or operator signature can claim one. A host may reserve a namespace of its own in turn (`com.acme.*`, say) for the standing services it runs on its own machines; a name under that prefix is honoured only on a template signed by that host's key. The prefix is the claim of authority and the signature is its proof, and the two must agree or the name is refused. What the operator does is take such a signed template (the maintainers' or the host's) and adapt their own version of it, filling the fields it leaves mutable, inheriting the reserved name and the rest exactly as signed, and enable it, the enablement being the grant of the host foothold. The vouching is layered to match: the tier that owns the namespace signs the reserved shape, and the operator stands up this instance of it. The shared thing that many confinements lean on is held to a higher bar than the private thing one workload holds: signed, at its origin, by the tier that owns the name, and bounded, where the operator varies it, to the blanks its author left open.

The residual is the other face of the same permanence. A standing provider that is compromised is compromised for every consumer that reaches it, for as long as it stays up: a wider blast than an ephemeral sibling's, whose damage ends with its task. The mesh does not erase that exposure; it replaces a worse one. Without it, the display, the bus, the cache are unconfined host code, and an application reaches them at the user's full authority: a compromise anywhere in that chain is host-wide. With it, those services are themselves confined to the one host leg each was granted, and the application reaches them through a declared, brokered, deny-by-default seam; a compromise is bounded to a provider that holds little, and to the consumers that declared they would trust it. The exposure is named, narrowed, and in view: far smaller than the ambient reach it replaces, and never pretended away.

With the mesh, the resource classes are complete. Every kind of thing a workload touches (its files, what it may run, the network, the local services of its session, the images it runs inside, the siblings it spawns, and now the capabilities it shares with other confinements) has been answered by the same two limbs and the same

handful of principles. The mesh comes last because it is the one exception to the isolation all the others build, and it earns the exception by holding to the rules they established: declared, not authored; brokered, not ambient; deny-by-default; and narrowed to the single seam the operator opened. What remains, in the chapters that follow, is not another resource but the grant itself: how a policy is written, composed, signed, and made to fail safe, the declared authority every one of these chapters has taken as given.

PART III

The declared grant

12. Defining confinement

The chapters to here have described what each boundary does. None has shown how an operator asks for one. That is the policy: the artefact in which a confinement is declared, signed, and handed to the framework to enforce. This part of the book is about the policy: how it is written, how it becomes the thing that enforces, and what crosses the boundaries it draws. Because a policy is a written thing, this is the first chapter that shows the writing.

What it shows first is how little of it there is.

12.1 Intent, not machinery

A confinement could be specified in full every time: every path denied, every default set, every floor restated, for every kennel. It is not. An operator's policy declares the intent particular to one kennel and inherits everything else from a template someone signed. A complete working policy for a coding agent is about this long:

```
template = "ai-coding-strict@v4"
name = "myproj-ai"

[[fs.read.add]]
path = "~/projects/myproj/**"
reason = "the project I am working on"

[[net.allow.add]]
name = "api.anthropic.com"
ports = [443]
reason = "the model API this agent calls"
threats.exposed = ["T1.8"]
```

Two grants and a name. The project this agent may read, the one endpoint it may reach: the facts only this operator knows. Everything that makes the kennel safe (the categorical denials, the empty world, the non-escalation floor, the deny-by-default on every resource) is in the template named on the first line, written and

12.2 Deltas carry their reason

signed by someone who maintains it, inherited here without being restated. The operator adapts a confinement; they do not compose one from nothing. It is the same discipline the delegation and mesh chapters turned on workloads and services, turned now on the operator: fill the blanks of a signed thing, do not author the whole.

12.2 Deltas carry their reason

What the operator writes are deltas: changes against the inherited policy. There are a few kinds: add an entry to a granted set, remove one the template allowed, override a single value, or (for a template author) mark a rule so that no policy downstream may remove it. The two add blocks above are the common case; the rest are rarer and, by design, more conspicuous.

Every delta must say why (tell-me-why). The reason field is not decoration and not optional: a policy that omits it does not compile. A grant without a reason is a grant nobody will understand in six months, and the design refuses to accept one. The reason travels with the delta into the audit log, the risk report, and the review diff, so the question *why can this agent reach that?* always has an answer attached to the grant itself rather than living in someone's memory.

The rarer the delta, the louder it reads. Adding a project path is unremarkable. Removing one of the template's default denials is not. It is the operator deliberately lowering a guard the template raised, and it shows up in the diff as exactly that:

```
[[fs.deny.remove]]
path = "-/.config/git/**"
reason = "this workflow needs the user's git config; accepted exposure"
threats.exposed = ["T1.1"]
```

The grant names the threat it accepts. An operator who writes this has not slipped past a default; they have signed their name to crossing it, in a line a reviewer reading the diff cannot miss.

12.3 Inheritance and the floor

The template the operator named is itself a short policy, written as deltas against a template above it, up to a root that grants nothing at all. The chain is linear by rule: each template extends exactly one parent, and the language forbids the branching structures that would make resolution ambiguous, so there is never a question of which of two inherited values wins:

```

base-confined@v1      (maintainer) grants nothing: deny-by-default on every
                        resource, the categorical denials, the non-escalation floor.
|
ai-coding-for-company  (host) adds the shape the organisation wants for a coding
                        agent: the tools it may run, the registries it may reach,
                        the secret files it scrubs.
|
my-coding-project-x    (operator) the leaf: this project, this endpoint.

```

The layers tend to fall along three tiers. The project's maintainers ship the root and the general templates; the host an operator runs on extends those with what the organisation requires; the operator writes the leaf with the facts only they hold. Each tier authors at its own level and signs with its own key, so a single effective policy is composed of links that were written and signed by different hands (the maintainer's root, the host's baseline, the operator's leaf), and resolving the chain verifies each link against the tier that signed it. The tiers are a convention of authorship, not a fixed depth: an operator may extend a maintainer template with no host in between, or a host may stack several of its own. What holds in every case is the shape: one parent per link, signed by whoever wrote it.

Read down the chain, a confinement is assembled from the most general to the most specific, each layer adding only what it alone knows. Read up it, every kennel traces back to the same root that begins by denying everything, so deny-by-default is not a setting a policy chooses but the floor every policy is built up from.

Some of that floor cannot be lowered. A template may mark a rule as an invariant, and an invariant propagates down every chain beneath it and cannot be removed by any delta, with any reason, however far downstream:

```

[[net.deny.invariant]]
cidr = "169.254.169.254/32"
reason = "the cloud metadata endpoint; never reachable from a kennel"

```

A leaf policy can remove an ordinary default deny (loudly, as above) but it cannot remove this one, and the compiler rejects the attempt rather than honour it. This is the floor of the resource chapters expressed in the language: the categorical denials that hold regardless of policy are invariants no policy can weaken, and the language draws the line between a default that may be crossed with a reason and a floor that may not be crossed at all.

One more limit keeps that floor meaningful, and it needs stating with care: a leaf policy is not, by itself, a template. An operator's adaptation confines their own kennel; it does not silently become a baseline other operators inherit from. This takes nothing from the operator. Templates are written and signed at all three tiers (by

the project's maintainers, by the host, and by the operator themselves), so an operator who wants a reusable baseline authors and signs one exactly as the other tiers author theirs. The operator is one of those who define confinement, not someone the framework holds at arm's length. The design confines the workload, never the operator who defines the confinement; a framework that reached to confine its own operator would only be bypassed, as the premise held from the first chapter. What the rule prevents is the accident: a one-off leaf, never meant as a baseline, copied and extended until a sprawl of unmaintained policies has drifted from the floor they all started above. A baseline is made deliberately (authored, signed, versioned) or it is not a baseline.

A policy, then, is a small and reasoned declaration: a handful of grants the operator alone can supply, written as deltas that each say why, over a deep inheritance of confinement the operator never restates and a floor the operator cannot lower. It reads as intent because the machinery is elsewhere: inherited, signed, and out of the operator's hands. What it does not yet explain is how this written intent becomes the thing that enforces: how the chain, the deltas, the signatures, and the floor are resolved into something a kennel can be held to, and what happens when any of it does not check out. That is the next chapter.

13. The settled policy

The policy from the previous chapter is what an operator writes. It is not what enforces. Between the two is a step skipped over until now: the written policy (with its inheritance chain, its deltas, its includes, and the signatures on each) has to be resolved into something a kernel can be held to, and that resolution is a great deal of fallible, security-critical work to be doing at the moment a workload starts. So it is not done then. The authored artefact and the enforced artefact are different things, and the work of turning one into the other happens once, in advance, at a step called compilation.

This is not novel; it is how the systems Kernel is measured against already work. AppArmor authors a text profile and compiles it into a binary the kernel loads, and the text is never consulted again at enforcement. SELinux compiles a monolithic binary policy from its source modules. The authored form is for people to read and reason about; the enforced form is for the machine to apply; and the compile step between them is where the expensive and dangerous parts are paid for, deliberately and once.

13.1 What compilation produces

Compilation takes a leaf policy and produces a settled policy: a single flat document in which the inheritance chain has been folded down, the includes merged, the deltas applied, and every source signature and pin verified. It carries no template reference, no include, no delta operator: nothing left to resolve. Only the final effective rules remain, and they are signed as a unit by whoever compiled them.

Everything hard happens here. The chain is walked from the leaf to the root and every link's signature checked against the tier entitled to have signed it; the includes are merged and any conflict between them is refused rather than guessed; the deltas are applied; the framework and template invariants are validated; the threat tags are checked against the catalogue; and the parts of the policy fixed for the whole installation are filled in. The result records its own provenance (every input that went into it, named by the key that signed it), so the settled policy can

13.2 The signature and the floor

be audited later without the templates it came from being present at all. What a kennel starts from, then, is not a policy to resolve but a policy already resolved: read, checked once, and frozen.

Who compiles a settled policy is not fixed to one tier. The common path has the operator compile their own leaf, but nothing obliges the tiers above to ship only adaptable templates. A maintainer or host can author a complete confinement, compile it, and publish it settled, ready to run with no leaf to write, beside the templates it offers for adaptation. The operator then has two shapes to choose between: inherit a template and write a leaf that adapts it, or run a finished policy a tier already baked and signed. Both are settled policies the runtime accepts on one signature, and either way the floor under the result is the framework's to re-assert at spawn, so running a policy baked upstream is no more an act of faith than running one's own.

13.2 The signature and the floor

A settled policy exists to be trusted on the strength of a single signature, and that is the larger half of what it buys. Handed one, the runtime does not re-walk the inheritance chain, re-apply the deltas, re-merge the includes, or re-verify the many source signatures that went into it. It verifies one signature (the compiling authority's, over the whole flat artefact) and on the strength of that accepts the entire body of the policy as resolved correctly. All the fallible work of resolution was done once, at compile; the signature is the runtime's reason to believe it was done right. The runtime does not re-argue the policy. It checks who vouched for it.

There is exactly one thing the signature is not trusted for, and it is the floor. A signature says a key the system trusts vouched for this artefact; it does not say the artefact is a valid kennel, because those are different claims. The structural guarantees that make a kennel a kennel (that privilege cannot escalate through it, that the categorical denials hold, that the home directory is shadowed, that the meta-data endpoint is unreachable) are the framework's guarantees, not the signer's. A trusted key vouching for a policy is no reason to believe the policy upholds them, because the key's authority is over what the operator may grant, not over what the framework must always deny.

So the framework re-asserts its own invariants directly, at the moment of every spawn, after the one signature is verified, regardless of the signature, and regardless of any claim the artefact carries about having been checked already. This is the single, deliberate exception to an otherwise complete reliance on the signature: the runtime trusts the signer for the whole of the policy except the part the signer was never entitled to weaken. The check is cheap, a handful of structural assertions, and it makes one guarantee unconditional: no policy, however it was

produced and whatever key signed it, can disable the protections that define a kennel (`reference-monitor`). A compromised compiling authority, a mistaken template, a forged claim of validity: none reaches the workload, because the last thing checked before the workload starts is not the policy's provenance but the policy's floor.

13.3 Fail-closed

Everywhere along this path, the response to something not checking out is the same: stop, and run nothing. A policy that does not parse is rejected before any signature is examined: there is no verifying the integrity of bytes that have no structure, so a malformed artefact never reaches the part of the system that would trust a signature on it. A signature that does not verify, an include that conflicts with another, a reference whose bytes have changed since they were pinned, an invariant a delta tried to weaken, a placeholder left unfilled at spawn: each refuses the kennel outright. No workload starts on a policy that failed a check, and no partial kennel is built from one that failed halfway; the failure is categorical, and it names what failed.

There is no degraded mode, and that absence is deliberate. A confinement that falls back to something weaker when it cannot apply what was asked is a confinement an attacker works to make fail. A kennel that cannot be resolved to a valid settled policy, or whose settled policy cannot be verified and floored at run, does not come up at all (`refuse-to-start`). The safe state when something is wrong is the empty one (nothing running), and the system holds to it rather than start a workload under a confinement it is unsure of: fail-safe defaults [SaltzerSchroeder], applied to starting and not only to granting.

One more property keeps the enforced artefact honest. A few grants are derived rather than written (a writable path implies a readable one, and so on), but the derivation is done at compile and written into the settled policy explicitly, never left to happen invisibly at run. What a kennel enforces is exactly what its settled policy says, down to the grants the operator never typed, so the enforced reality can always be reconciled with the authored intent by reading the artefact. Nothing the operator did not intend appears, and nothing they did intend is silently widened.

13.4 Authored in one place, enforced in another

Putting all the resolution at compile time does more than keep the spawn fast; it shrinks what the running system must be trusted to do. The path that starts a kennel links none of the machinery that resolves a policy: no template parsing, no chain-walking, no include merging, no delta application, no verifying of source signatures. All of that crossed its gate at compile and need never run again. What the

spawn path does is verify one signature, re-assert the floor, fill in the per-instance values, and build the kennel: a small, fixed surface that does not grow with the complexity of the policy it enforces.

This is what lets confinement be authored in one place and enforced in another. An organisation can compile its policies centrally (where the templates, the keys, and the review live) and ship only the signed settled artefacts to the machines that run workloads. Those machines need not hold a single template or run the resolver; their entire policy-trust surface is one signature check and the floor re-assertion. The operator who writes a leaf on their own machine and the fleet that enforces a settled policy pushed from a central repository are doing the same thing at different scales, and in both the dangerous part was paid for before the workload ever started.

The life of a policy, then, is a sequence of refusals as much as a sequence of steps. It is authored as intent, compiled once with every check applied and any failure stopping the whole, settled into a flat artefact that carries its own provenance, and run behind a single signature and a floor the framework re-asserts for itself, trusting no signer to have left it intact. What this leaves unanswered is the question the signatures keep raising: whose keys does the system trust, for what, and why is that the right division? Which tier may vouch for a floor, which may only grant within one, and which may sign nothing that others must rely on: that is the consent model the whole design rests on, and it is the last thing to set down.

14. Trust and consent

The grant has been written and the grant has been enforced, and both rest on signatures. A signature is only ever as meaningful as the trust behind the key that made it, so the trust itself has to be set down: the hierarchy it forms, what is allowed to cross its boundaries, and who is entitled to vouch for what. It is also where the reference monitor's third property, that the thing to be trusted be small enough to trust, finally comes home, now that everything it guards has been described.

14.1 The trust root

At the top of the hierarchy is not the framework but the user. The default context (the user's ordinary, unconfined shell, with the full authority of their account) is the trust root, and every kennel sits beneath it (`split-the-uid`). The framework that builds and mediates those kennels is not above the user in any sense that matters: it can do nothing in the user's name that the user could not do from their own shell, it owns no authority the user lacks, and it runs, like everything else, as the user. What it has that a kennel does not is the policy decisions, the mediation, the record: it sits higher than the things it confines, and level with the hand that configures it.

And it is bounded by consent at every turn. The user installs the framework, writes or adopts its policies, enables its services, and can switch the whole of it off. It does not try to confine the user, and could not if it tried: a confinement framework that set out to restrain the very context that installs and configures it would simply be removed, and a design that depends on restraining its own trust root has misunderstood where the trust lies. So the framework draws its boundaries around the processes the user chose to confine, and around nothing else. An adversary already in the default context is not a kennel that has escaped; it is the user, by assumption, and there is nothing left to confine them with, which is exactly why the line is drawn where it is, with the user on the trusted side of it.

14.2 What crosses

Because the boundary is built by construction, what crosses it is short to list, and the list is exactly what the operator declared (*construction-by-absence*). Into a kennel goes the invocation that started it, the environment the policy synthesised, the constructed view of the filesystem, the standard streams where they were granted, and a working directory inside the granted paths. Out of a kennel comes what a parent is owed: the exit status and signals to the process that launched it, the standard output and error, the audit record written through the framework rather than by the workload's own hand, and whatever the workload wrote into the paths it was granted to write. That is the whole of it.

Everything else does not cross, and does not have to be enumerated to be denied. A workload does not reach the user's session bus, raise a notification, touch the clipboard, send input to another window, or signal a process outside its own boundary, not because each of these was thought of and forbidden, but because none of them was constructed into the kennel's world, and what is not there cannot be crossed to. The consent model is legible for the same reason: an operator reads what crosses by reading what they declared, and may take everything they did not mention as not crossing. Between two sibling kennels the list is shorter still: nothing crosses by default, the two being mutually invisible until a policy on both sides declares a seam between them.

14.3 What the tiers are for

Trust enters the system as signatures, but the signatures carry less than it might seem, because the thing that most needs securing does not depend on them. The floor (the invariants that make a kennel a kennel) is re-asserted at every spawn regardless of who signed the policy, or whether it was signed at all (Chapter 13). It secures itself (*reference-monitor*). A settled run-policy can only narrow within a floor it cannot move and runs with the operator's own authority, so an operator may sign their own without granting any escalation, and there is nothing to withhold. With the floor off their shoulders, what the signatures are left to do is mostly a matter of practice, with a single exception.

The maintainers ship a set of sane templates (a base that denies everything, and useful shapes built on it) because a framework that offered a blank deny-everything and a manual would be unusable, and an unusable safety tool is none. An operator picks one and fills in their specifics, or writes their own; nothing forbids a baseline of one's own on a machine whose trust one controls.

Which signatures a machine treats as baselines is set by whoever controls its trust configuration: the operator on their own machine, the organisation on a managed

fleet, where a personal key is not a system key and a user cannot quietly swap a mandated baseline for one of their own. That is what lets a company require a confinement for a task and have it stick, the way device management and audit and workplace policy make other obligations stick: governance the signatures support rather than impose.

One thing they do impose. A reserved name (the kind a kennel resolves against to reach a standing service) is claimable only by the tier that owns it: the project's own names, under `org.projectkennel.*`, by the maintainers; a host's namespace by that host. Here the signature is not serving a convenience or backing a rule that something else enforces; it is the authority itself, because a name others trust to resolve to a particular provider is worth nothing unless it is anchored to whoever is entitled to answer it. The carve-out is deliberately small.

14.4 Small enough to trust

All of this (the hierarchy, the crossings, the signatures) exists to be trusted, and a thing to be trusted must be small enough to be understood. This is the reference monitor's third property, and it is where the design's recurring question, how can I do less, stops being a disposition and becomes a discipline (`do-less`). The trusted base is what has to be audited, so every part of it must earn its place: a facade is admitted only where construction cannot reach the threat, and each is answerable to the specific threat it closes rather than added because it might help. What can be left out is left out, because what is not in the base is not in the audit.

The honest cost of the parts that remain is smaller than it first appears. A facade does parse adversary-supplied protocol, on every message, for the life of the kennel: the obvious objection to interposition, and a real one. But the facade sits on the untrusted side of the boundary and holds no authority. It turns a hostile request into a typed transaction and does nothing else; the decision on that transaction is the monitor's, made on an identity the workload cannot forge, and any privileged effect is carried out by a policy-free actor doing only what the monitor already authorised. The parser of hostile input is therefore outside the thing that must be trusted, present in the system but not in the base whose correctness the whole rests on.

That accounting is clean for the ephemeral case and owes a concession for the standing one. The actor behind a facade is policy-free but not effect-free: it carries out the authorised transaction, and a standing provider carries the one leg into the host its job requires (the real bus, the real display) for the whole of its life, not for a single brokered call. A compromised provider does not forge a decision, but it holds that granted leg and can act through it within the authority the grant already gave, which is the standing-provider residual named where the mesh introduces

it (T3.10). The parser is outside the base; the standing foothold is not removed, only confined to the one leg and made answerable for it. The two sections state the same fact at different altitudes, and the honest reading is the narrower: interposition moves the hostile parser out of the trusted base and bounds the provider's reach, it does not reduce a standing provider's granted foothold to nothing.

The trust root and the tiers above it have stood behind every signed thing in the design, the authorities that vouch for what an operator may inherit, yet the material that vouching is made of has stayed implicit. What a key is, what its presence and absence mean, and why the framework mints none of it, is the substrate the rest has rested on without naming, and the next thing to set down.

15. Keys

Every chapter to here has leaned on a word it did not define. A policy is signed; the runtime checks who vouched for it rather than re-arguing it; the trust root is the operator, and the tiers above the operator sign the templates an operator inherits. Behind each of those is a key, and the volume has not said where a key's authority comes from, what it means for one to be trusted, or what changes when one is taken away. The trust material is the last substrate to set down, and it is the smallest, because the rule that runs through the rest of the design runs through it unchanged: what is trusted is constructed from what is present, and withdrawn by absence.

15.1 Trust is placed, not minted

A kennel runs code the operator has not chosen to trust, and the framework holds to a matching discipline about trust of its own: it will mint a keypair freely, and it mints no trust at all. Generating a keypair is mechanical and confers nothing; what confers authority is where the public half is placed. A key is trusted to vouch for a layer of policy because someone with standing put it where that layer's keys are read from, for no other reason than that it is there (`authentication-never-attestation`).

This is the construction limb applied to the trust material rather than to a resource (`construction-by-absence`). A view of the filesystem holds the paths granted into it and no others; a tier's authority holds the keys placed into it and no others. What is trusted is the exact set of keys present, not everything a denylist has not yet caught. There is no standing list of bad keys, because there is nothing a list would add: a key absent from the place a tier's keys are read is already without authority, indistinguishable from one that never existed. The gains are absence's usual ones. Nothing has to be enumerated or kept current, and an error of trust is a key visibly present or visibly missing, never a stale entry on a revocation list that was meant to catch it and did not.

The framework leans into this rather than standing back from it. The ordinary case

is one operator on one machine who needs a single key to begin, and the framework will generate that key and place it in the operator's own location as part of setting up, so a new user authors and signs at once instead of first learning to make and install a key by hand. The key is no special thing: an ordinary public-key identity in the platform's standard format, not a type the framework invented, so the operator receives something they may already know how to hold and inspect, and the framework has one fewer thing to define (*do-less*). It confers no trust by doing so; it spares the operator a mechanical step in trusting themselves, since a generated and placed operator key is the operator's own signing identity, stood up by construction in the one tier whose authority is theirs to hold. What it will not do is the rest: pronounce another party's key good, broker trust between two parties who do not already share it, or become the authority that decides which keys a tier should hold. Those are attestation, and a kennel withholds attestation by definition.

15.2 The hierarchy of locations

The tiers are the ones the policy model already named: a maintainer who ships a base, a host who adapts it for a fleet, an operator who writes the leaf. Each tier has a place its keys are read from, and the places stand in the same order as the tiers. When a settled policy is checked, each link in its inheritance chain is verified against the place belonging to the tier that signed that link: a maintainer's base against the maintainer's keys, a host's adaptation against the host's, an operator's leaf against the operator's. A key in the operator's place cannot vouch for a layer claiming a maintainer's authority. Where a key sits is the scope of what it may sign.

A place holds any number of keys, and the number may be zero. Several keys in one place are several independent signers for that tier, any one of which verifies a layer it signed: a tier with three keys present is three holders who may each vouch, not a quorum that must agree among them. This is the keys-dimension of multi-operator, and it is free, because a set of trusted keys was always a set. Who may add a key to a place, and how holders are scoped against one another, is a fuller delegation question the design leaves open; the plain fact that a tier can carry many signers, each admitted by presence and withdrawn by absence, falls out of the model with nothing added.

Zero keys is the case worth stating, because it is muteness and not error. A place with no keys is a tier that signs nothing and vouches for nothing; it is no fault and raises no failure on its own. A chain fails only where it required a signature the empty place would have supplied, and there it fails as every unmet requirement in this design fails, by refusing to start rather than running degraded (*refuse-to-start*). Emptiness at the host tier is the expected shape rather than the exception: most machines carry a maintainer's base and an operator's leaves with

nothing between them, and the host's place stays empty until an organisation has reason to fill it.

15.3 Admission, not termination

A key is consulted at one moment, when a kennel is constructed and its policy verified. It is not consulted again while the kennel runs, because a running kennel re-reads no policy and re-verifies no signature; its world was settled at the start and does not shift under it. That is what gives removal its exact meaning. Taking a key out of its place reaches into nothing already running. It changes one thing: the next construction that would have relied on that key fails to verify, and refuses to start.

The shape to hold is locking an account rather than killing its processes. Locking an account stops the next login and leaves the open sessions alone; removing a key stops the next kennel and leaves the running ones alone. They are different acts with different blast radii, and treating one as the other is the usual mistake. Removal is the quiet, construction-time act: it withdraws the authority to make new kennels under that key, and says nothing about the kennels the key has already admitted.

When a running kennel does have to be stopped, that is the other act, and the design keeps the two apart. A kennel is terminated from outside by an explicit command, the way any supervised process is ended by something above it, and that lever belongs to the tool that runs kennels, not to the trust material. Keys govern admission and only admission. Whether a kennel may start is the key model's question; whether a running one must stop is answered by the mechanism that supervises kennels, not by the one that vouches for their policies.

15.4 Keys over time

A design that builds trust from present keys and consults them only at construction has, almost in passing, said most of what there is to say about policy over time. The machinery a longer-lived system usually accretes (a register of retired keys, a deprecation state carried on templates, tooling to march a fleet from one version to the next) is machinery this design does not build, and in each case for the same reason: the model covers the ground without it.

A template version owes nothing to the versions before it. It is a complete statement at its place in a chain, not a difference against its predecessor, and a newer version may be stricter than an older one in any way it chooses, up to adding a categorical denial the older one did not carry. Because a settled policy is immutable once compiled, none of this reaches a kennel by itself: a settled policy was resolved against

the versions it named, carries its own signature, and is untouched by any later version of anything it drew from. Change is always deliberate, a re-compilation against the newer material that yields a new settled policy for the operator to sign. Nothing settled moves, so nothing drifts.

The version is named in the open: a leaf inherits not from `base-confined` but from `base-confined@v1`, and the version is a fixed part of what the name points at. That makes the three things an author can do to a published version distinct. Editing `@v1` in place is the unsafe one, because every policy that named it resolves against the changed thing on its next compile, and a change that tightened a rule or moved a field surfaces as a compile failure in a policy that was sound the day before. Publishing `@v2` beside it touches nothing already named: `@v1` still resolves, and adopting `@v2` is a migration done one policy at a time, by re-pointing and re-compiling on the operator's own schedule. Removing `@v1` behaves like every removal in this design: the name stops resolving, and the next policy that tries to recompile against it does not find it and refuses, exactly as a construction that reached for an absent key refuses. A published version is best treated as immutable, then: add beside it, retire by removing it, and do not edit it under the policies that already trust it by name.

Where that deliberate re-compilation meets a floor that has risen, the design's preference for loudness finishes the job. A newer version that raises the floor will refuse to produce a kennel for a leaf that crossed the new line, and it refuses at compile, in front of the operator performing the upgrade, named as the breaking change it is (`refuse-to-start`). A raised floor is never a quiet narrowing found out later; it is a re-compilation that halts and says what it will no longer allow. An operator who wants the older, looser behaviour still has it, in the policy they already settled and signed, until they choose to move.

Deprecation, then, is not a state a key or a version carries. It is the absence of the key you would have used. A key does not know it is being retired and is not marked as retired; the discipline of signing is what steers an author to the right key, and the only hard lever is the one already described, removing the key so the next construction that reaches for it refuses. The design keeps no record of what is old. It keeps what is present, and lets absence carry the meaning it carries everywhere else in it (`do-less`).

One thread has run through every chapter without being drawn out on its own: each time the monitor decides, it can record what it decided. That account, and what it may and may not contain, is the last piece of the design to set down.

16. Audits

A reference monitor decides. By the same property that lets it decide every use, it can record every decision it makes. Auditing is not a feature bolted onto the design; it is what complete mediation looks like written down. A boundary that sees every crossing can keep an account of them, and a boundary worth trusting keeps that account on the side the workload cannot reach.

16.1 The record of mediation

Mediation happens at the interposition limb, where a use is presented and a decision returned (*interpose-as-transaction*). That is the point at which there is something to record: a request was made, a rule applied, an answer given. The construction limb leaves little to log, because what was made absent is never reached and so never refused; a workload does not attempt what is not in its world (*construction-by-absence*). The audit is mostly the diary of interposition, the running account of the decisions the monitor was asked to make and the answers it gave.

Because mediation is complete, the account can be complete in the same sense (*reference-monitor*). There is no mediated decision that escapes the possibility of being recorded, since recording is one more step the monitor takes at a point it already controls. The audit inherits the reach of the thing it watches. A boundary that mediated only some uses could account for only some; a boundary that mediates all of them can account for all of them, and the completeness of the record is not a separate achievement but the same one seen from another angle.

16.2 Written by the trusted side

The account is worth keeping only if the subject of it cannot edit it. So the framework writes the audit, and the workload never does. The workload is what the record is about, not what produces it. Every event is emitted on the trusted side, by the monitor that made the decision, not by the process the decision was made against.

This closes the obvious attacks before they begin. A workload cannot forge an event, because it does not hold the pen. It cannot suppress one, because it does not control the writer. And by default it cannot even read the account kept of it (T1.1), because the audit is not placed in its constructed view: the log is simply absent from the workload's world, in the same way the first limb makes other things absent. A workload that tried to learn what had been observed of it, or to write a misleading entry into the record (T2.2), would find nothing there to read and nothing to write to. The audit is tamperproof from the side that would have reason to tamper with it, which is the property the enforcer has and for the same reason: the thing under watch holds no authority over the watching.

16.3 Decisions, not contents

What the record holds is the shape of each decision and no more: which resource was reached for, which target, whether it was allowed or denied, at what time, in which kennel, and under which rule. The rule brings its reason with it, the same reason the policy was made to carry, so a denial does not merely say no; it says which line said no and why that line exists. The record is legible because the policy that produced it was made legible first (`tell-me-why`).

Denials are always kept, because a denial is the security event: something attempted a crossing and was stopped, and the operator needs to be able to ask what their workload tried to do that the boundary refused. Allowances are kept more sparingly. A working kennel produces a great many of them and most are the dull expected case, so the record summarises them rather than drowning the denials beneath them. How much is kept is the operator's to set, but the default leans toward signal.

One class of thing is never written, at any level. The audit does not record secrets, credentials, keys, or the contents of the files the kennel exists to keep in. It records that a secret-bearing file was read, never the secret on the line below. This is not an omission to be fixed by a verbose mode but a rule the design holds on purpose, because an audit that copied the protected payload into a second file would be a leak wearing the dress of accountability. The record is of the decision, never of the thing the decision protected. It says, in this, on the same side of the line the network chapter drew: the boundary accounts for the channel it mediates, not for the payload it was never reading.

16.4 A record made to be used

A record kept on the trusted side and never polluted with payload is worth building on, and the design treats it as an interface rather than as a heap of text. The

events have a stable, structured, documented shape, so that something downstream can read them years apart and rely on what it finds. That stability is what turns the account from a curiosity into a tool.

The plainest use is the daily one. An operator puzzled by a workload that cannot reach some service reads the denials and is told, in as many words, which rule refused the connection and why. Past that lie heavier uses the same record supports without being changed for them: a forensic reading of what a workload touched within some window; a compliance demonstration that a kennel never reached a resource it was forbidden, evidenced by the refusal of each attempt rather than by a promise; and the feedback into authoring, where the record of what a workload kept trying to do shows the operator where a grant is too tight or too loose. Defining confinement was described as where the reasons go. This is where they arrive.

16.5 What the record does not claim

Two honest limits close the account. The first is on the record's own integrity once it has left the framework's hands. The audit is append-only from the side that writes it, and the workload cannot touch it, but stronger guarantees against a compromised host, such as segments signed as they are written or an external transparency log, are a layer above this one. Kennel makes them possible without providing them itself (`do-less`). It yields a record trustworthy against the workload; sealing that record against the machine is a further thing, and a downstream one.

The second is on what a record of mediation can ever contain. It holds the crossings, not the reasoning behind them: what a workload reached for, never what it meant to do with what it reached. A monitor sees its boundary and accounts for it faithfully, and it sees nothing of the workload's inner life, which was never its to mediate or to know. The account is complete about the boundary and silent about the mind behind it, and it is honest in being both.

With that, the design is laid out. A kennel confines by constructing a small world and mediating the few seams it leaves; it declares that confinement in a signed and reasoned grant; it enforces the grant behind a floor no signature can lower; and it keeps an account of every decision on the side the workload cannot reach.

References

Works cited across Volume 1. Chapters cite by key — [Anderson], [Irvine] — and the keys resolve here. Each work is entered once.

[Anderson] James P. Anderson, *Computer Security Technology Planning Study*, ESD-TR-73-51, Vol. I, Air Force Electronic Systems Division, Hanscom AFB, Bedford, MA, October 1972. The reference monitor and its three requirements are set out in §4.1.1. <https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72.pdf>

[Irvine] Cynthia E. Irvine, “The Reference Monitor Concept as a Unifying Principle in Computer Security Education,” in Proc. IFIP TC11 WG 11.8 First World Conference on Information Security Education, 1999, pp. 27–37 (Naval Postgraduate School, June 1999). <https://apps.dtic.mil/sti/tr/pdf/ADA423529.pdf>

[SaltzerSchroeder] Jerome H. Saltzer and Michael D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, September 1975. Origin of the design principles, among them complete mediation, least privilege, economy of mechanism, and fail-safe defaults. <https://www.cs.virginia.edu/~evans/cs551/saltzer/>

[Schneider] Fred B. Schneider, “Enforceable Security Policies,” *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, February 2000. Characterizes the policies a monitor can enforce by watching execution — exactly the safety properties — with complete mediation and isolation among its conditions, and security automata as the specification. <https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf>